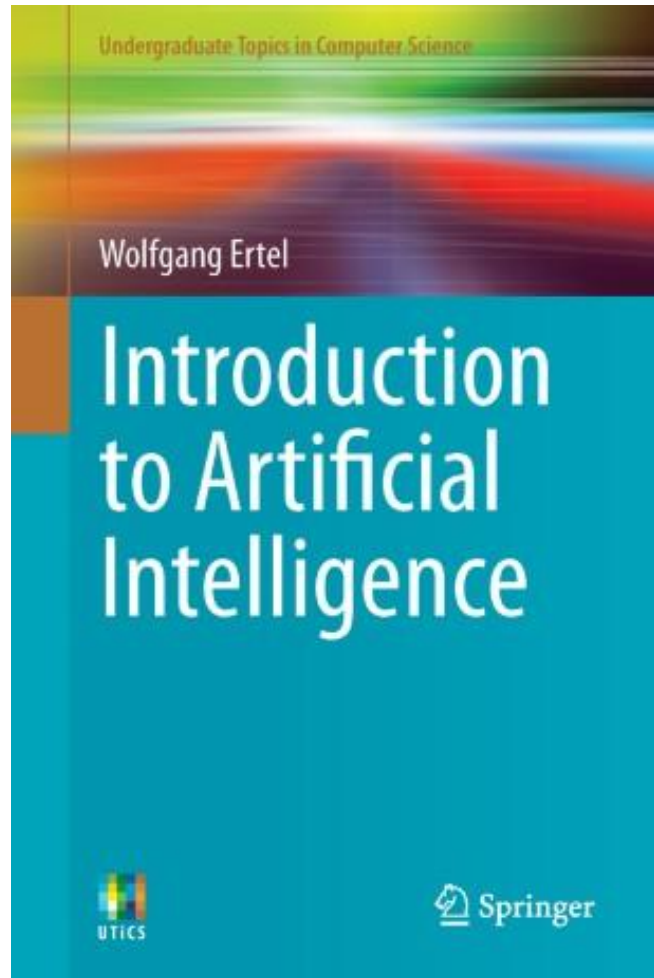


AI Fundamentals

Chapter 4 – Search, Games and Problem Solving



VIVES University of Applied Sciences
Bachelor in electronics-ICT



AI Fundamentals

Chapter 4: Search, Games and Problem Solving

- We would like to thank the following people for their contribution:
 - Stefaan Haspeslagh, VIVES
 - Yves Sagaert, VIVES
 - Andy Louwyck, VIVES
 - Prof. Danny De Schreye, KU Leuven

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

Contents

- 1. Introduction**
2. Uninformed Search
3. Heuristic Search
4. Games with opponents
5. Heuristic Evaluation Functions
6. State of the Art

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction

- The search for a solution in an **extremely large search tree** presents a problem for nearly all inference systems.
- From the **starting state** there are many possibilities for the **first inference step**. For each of these possibilities there are again many possibilities in the next step, and so on.
- An example of a limited search tree is illustrated in **Figure 4.1**. The tree was cut off at a **depth of 14** and has a solution in the leaf node marked by *.
- For realistic problems, the **branching factor** and **depth** of the first solution may become significantly bigger.
- URL: [vertakkingsfactor \(branching factor\)](#)
- URL: [search tree](#)

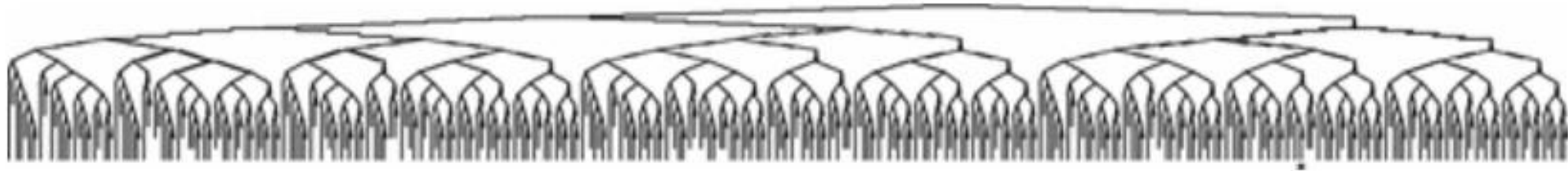


Figure 4.1: Limited search tree

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction

- Assume the **branching factor** is a constant equal to **30** and the first solution is at **depth 50**.
- The search tree has $30^{50} \approx 7.2 \cdot 10^{73}$ **leaf nodes**.
- But the number of inference steps is even bigger because not only every leaf node, but also every **inner node** of the tree corresponds to an inference step.
- Therefore we must add up the nodes over all levels and obtain the total number of nodes of the search tree

$$\sum_{d=0}^{50} 30^d = \frac{1 - 30^{51}}{1 - 30} = 7.4 \cdot 10^{73}$$

- which does not change the node count by much.
- URL: [Meetkundige reeks](#)
- Evidently, nearly all of the nodes of this search tree are on the **last** level.

1. Introduction

- But now back to the search tree with the $7.4 \cdot 10^{73}$ nodes.
- Assume we had 10.000 computers which can perform a billion inferences per second and that we could distribute the work over all of the computers with no cost.
- The total computation time for all $7.4 \cdot 10^{73}$ inferences would be approximately equal to
$$\frac{7.4 \cdot 10^{73} \text{ inferences}}{10.000 \cdot 10^9 \text{ inferences/s}} = 7.4 \cdot 10^{60} \text{ s} \cong 2.3 \cdot 10^{53} \text{ years}$$
- *which is about 10^{43} times as much time as the age of our universe.*
- By this simple thought exercise, we can quickly recognize that **there is no realistic chance of searching this kind of search space completely** with the means available to us in this world.
- Moreover, the assumptions related to the size of the search space were completely **realistic**. In **chess** for example, there are over **30** possible moves for a typical situation.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

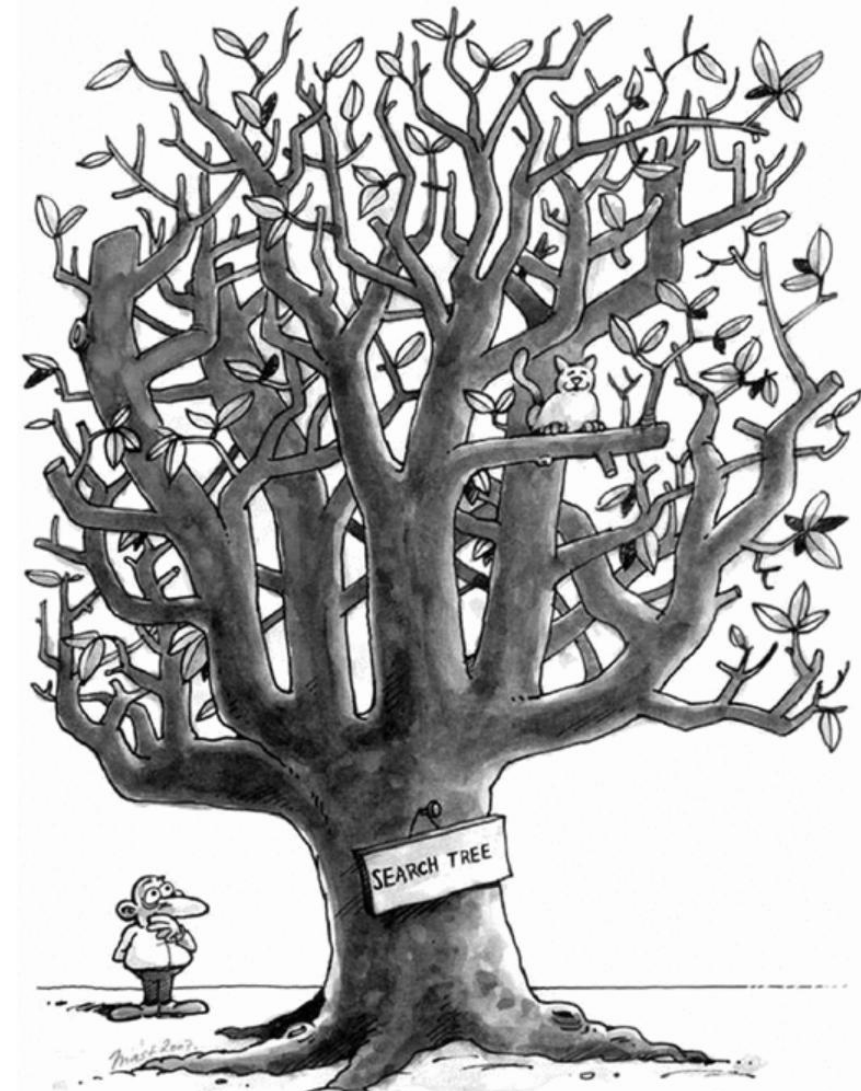
1. Introduction

- *How can it be then, that there are good chess players – and these days also good chess computers?*
- *How can it be that mathematicians find proofs for theorems in which the search space is even much bigger?*
- Evidently we humans use **intelligent strategies** which dramatically **reduce the search space**.
- The experienced **chess player**, just like the experienced **mathematician** will, by mere observation of the situation, immediately **rule out many actions as senseless**.
- Through this **experience**, he has the ability to evaluate various actions for their utility in reaching the goal.
- *Often a person will go by feel*. If one asks a mathematician how he found a proof, he may answer that the **intuition** came to him in a dream.
- In difficult cases, many **doctors** find a diagnosis purely by *feel*, based on known symptoms.
- Especially in difficult situations, there is often **no formal theory** for solution-finding that guarantees an optimal solution.

1. Introduction

- In everyday problems, such as the search for a runaway cat in **Figure 4.2**, *intuition* plays a big role.
- We will deal with this kind of **heuristic search method** and additionally describe processes with which computers can, similarly to humans, improve their **heuristic search strategies** by learning.
- First, however, we must understand how **uninformed search**, that is, *blindly trying out all possibilities*, works.

Figure 4.2: A heavily trimmed search tree – or “Where is my cat?”



AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction – example 1

- With the **8-puzzle**, a classic example for search algorithms, the various algorithms can be very visibly illustrated.
- Squares with the numbers 1 to 8 are distributed in a 3x3 matrix like the one in **Figure 4.3**.
- The goal is to reach a certain ordering of the squares, for example in **ascending order** by rows as represented in **Figure 4.3**.
- In each step a square can be moved **left**, **right**, **up**, or **down** into the **empty space**.
- The empty space therefore moves in the corresponding *opposite* direction.
- For analysis of the search space, it is convenient to always look at the possible movements of the **empty** field.
- URL: [8 Puzzle Problem](#)

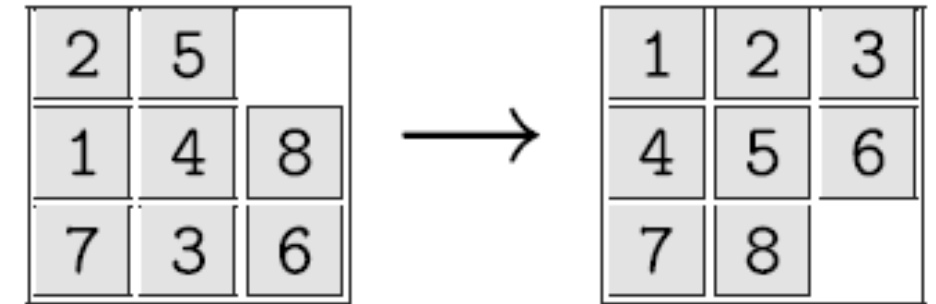


Figure 4.3: Possible starting and goal states of the 8-puzzle

Chapter 4: Search, Games and Problem Solving

- The **search tree** for a starting state is represented in **Figure 4.4**.
- *Bottom right* a **goal state** in **depth 3** is represented. To save space the other nodes at this level have been omitted. We can see that the **branching factor** alternates between **two**, **three** and **four**.

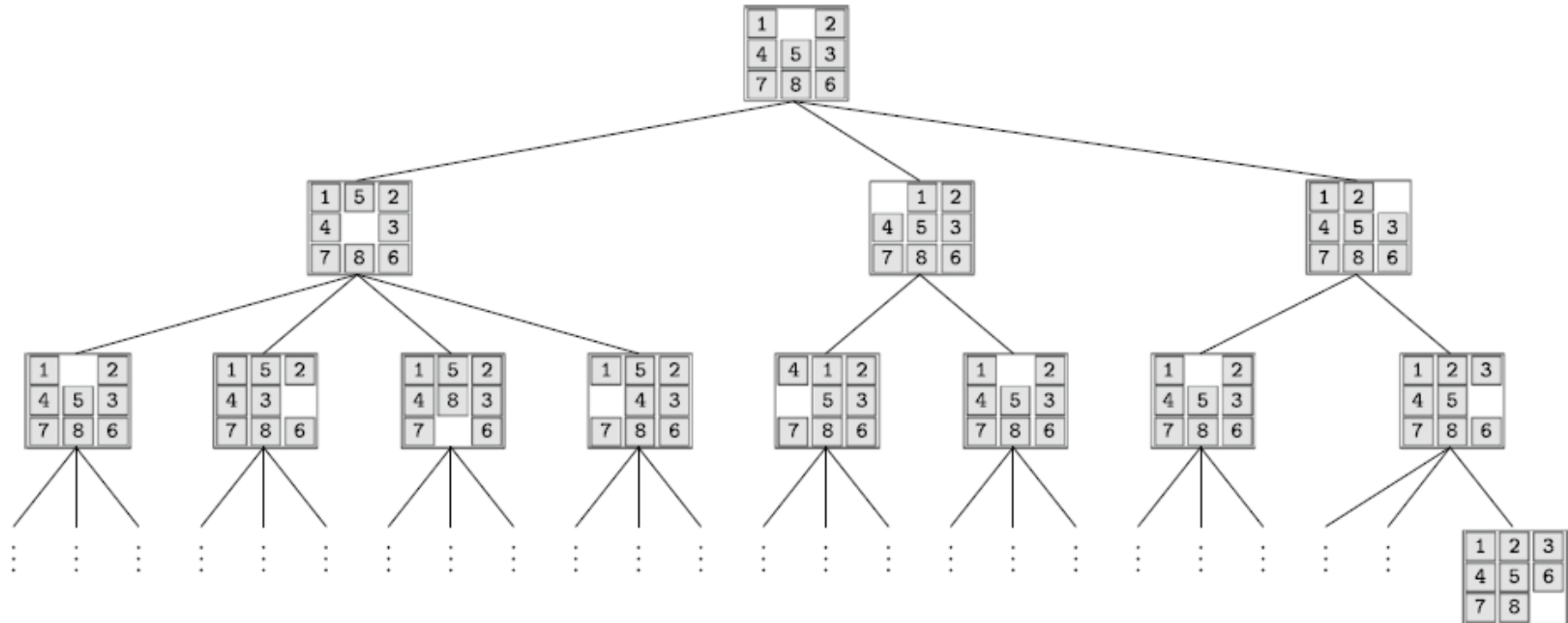


Figure 4.4: Search tree for the 8-puzzle.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction – example 1

- Averaged over **two** levels at a time, we obtain an **average branching factor** of $\sqrt{8} \approx 2.83$.
- We see that each state is **repeated multiple times** two levels deeper because in a simple **uninformed search**, every action can be *reversed* in the next step.

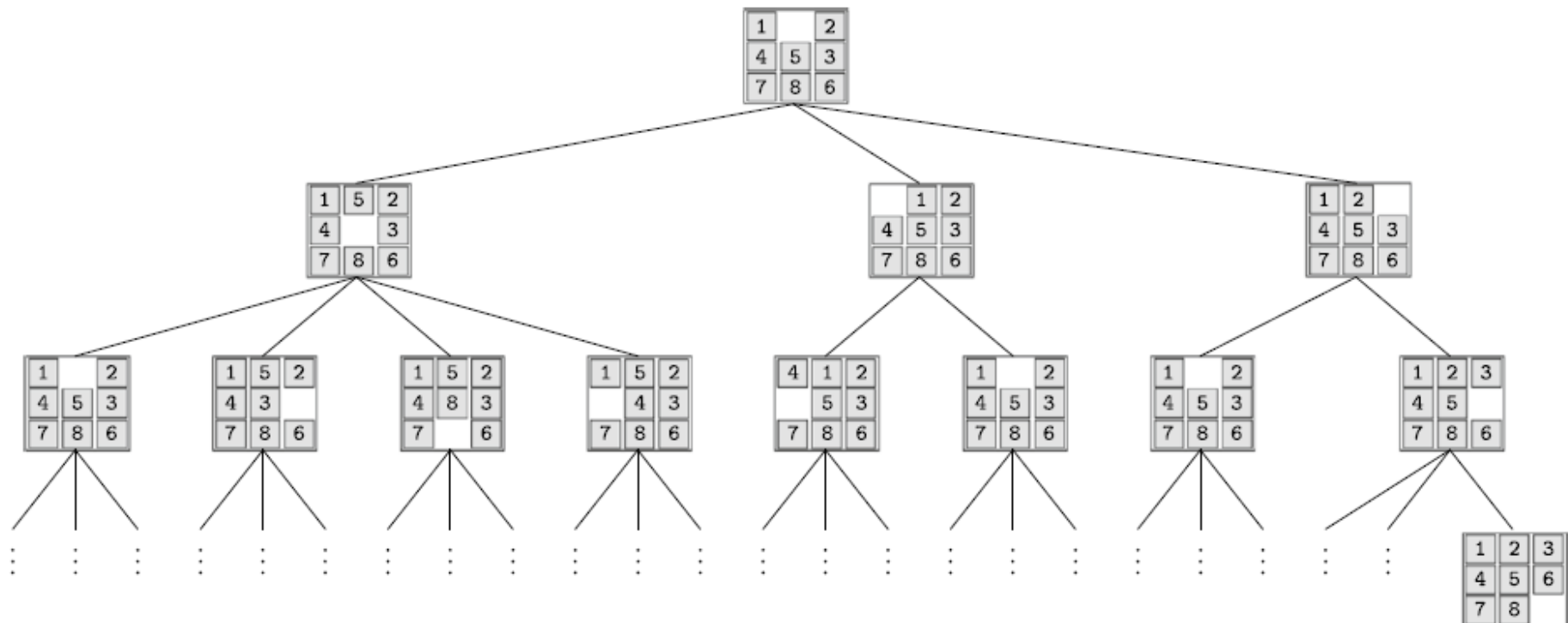


Figure 4.4: Search tree for the 8-puzzle.

AI

Ch

Chapter 4: Search, Games and Problem Solving

1. Introduction – example 1

- If we *disallow cycles of length 2* then for the **same starting state** we obtain the search tree represented in **Figure 4.5**. The **average branching factor** is reduced by about 1 and becomes **1.8**.
- For an 8-puzzle the **average branching factor** depends on the **starting state**.

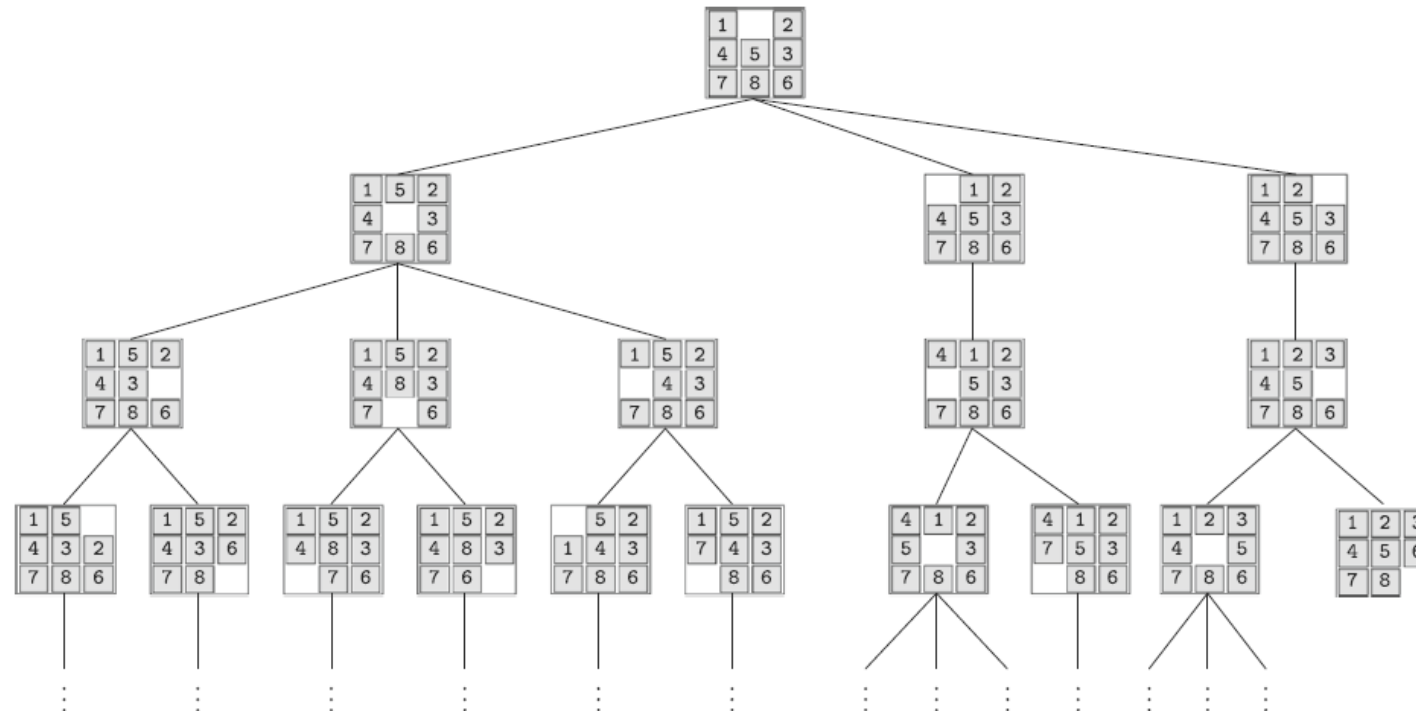


Figure 4.5: Search tree for the 8-puzzle without cycles of length 2

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction – example 1

- Before we begin describing the **search algorithms**, a few new terms are needed.
- We are dealing with *discrete* search problems here.
 - Being in state s , an action a_1 leads to a new state s' . Thus $s' = a_1(s)$
 - A different action may lead to state s'' , in other words: $s'' = a_2(s)$
- **Recursive** application of all possible actions to all states, beginning with the starting state, yields the **search tree**.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction – example 1

Definition 1 A search problem is defined by the following values

State: Description of the state of the world in which the search agent finds itself.

Starting state: The initial state in which the search agent is started.

Goal state: If the agent reaches a **goal state**, then it terminates and outputs a solution (if desired).

Actions: All of the agents allowed actions.

Solution: The path in the search tree from the starting state to the goal state.

Cost function: Assigns a cost value to every action, necessary for finding a cost-optimal solution.

State space: set of all states.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction – example 1

- Applied to the **8-puzzle**, we get:
- **State**: 3x3 matrix S with the values 1, 2, 3, 4, 5, 6, 7, 8 (once each) and one empty square.
- **Starting state**: An arbitrary state.
- **Goal state**: An arbitrary state, e.g. the state given in **Figure 4.3**.
- **Actions**: Movement of the **empty** square S_{ij} to the **left** (if $j \neq 1$), **right** (if $j \neq 3$), **up** (if $i \neq 1$), **down** (if $i \neq 3$).
- **Cost function**: The constant function 1, since all actions have **equal cost**.
- **State space**: The state space is degenerate in domains that are *mutually unreachable*. Thus there are *unsolvable* 8-puzzle problems.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction – example 1

- For analysis of the search algorithms, the following terms are needed:

Definition 2

- The number of successor states of a state s is called the **branching factor** $b(s)$, or b if the branching factor is **constant**.
- The **effective branching factor** for a tree of depth d with n total nodes is defined as the branching factor that a tree with **constant branching factor, equal depth** and **equal n** , would have.
- A search algorithm is called **complete** if it finds a solution for every **solvable** problem. If a **complete** search algorithm terminates without finding a solution, then the problem is **unsolvable**.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction – example 1

- For a given depth d and node count n , the **effective branching factor** can be calculated by solving the equation

$$n = \frac{b^{d+1} - 1}{b - 1} \quad [4.1]$$

- for b , because a tree with **constant branching factor** and depth d has a total of

$$n = \sum_{i=0}^d b^i = \frac{b^{d+1} - 1}{b - 1} \quad \text{nodes} \quad [4.2]$$

- For the practical application of search algorithms for **finite** search trees, the **last** level is especially important because of **Theorem1**.

Theorem 1

*For heavily branching **finite** search trees with a large **constant branching factor**, almost all nodes are on the **last level**.*

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction – example 2

- We are given a **map**, such as the one represented in **Figure 4.6**.
- **State**: A city as the current location of the traveler.
- **Starting state**: An arbitrary city.
- **Goal state**: An arbitrary city.
- **Actions**: Travel from the current city to a neighboring city.
- **Cost function**: The distance between the cities. Each action corresponds to an **edge** (= zijde/rand [NL]) in the graph with the distance as the weight.
- **State space**: All cities, that is, nodes of the graph.

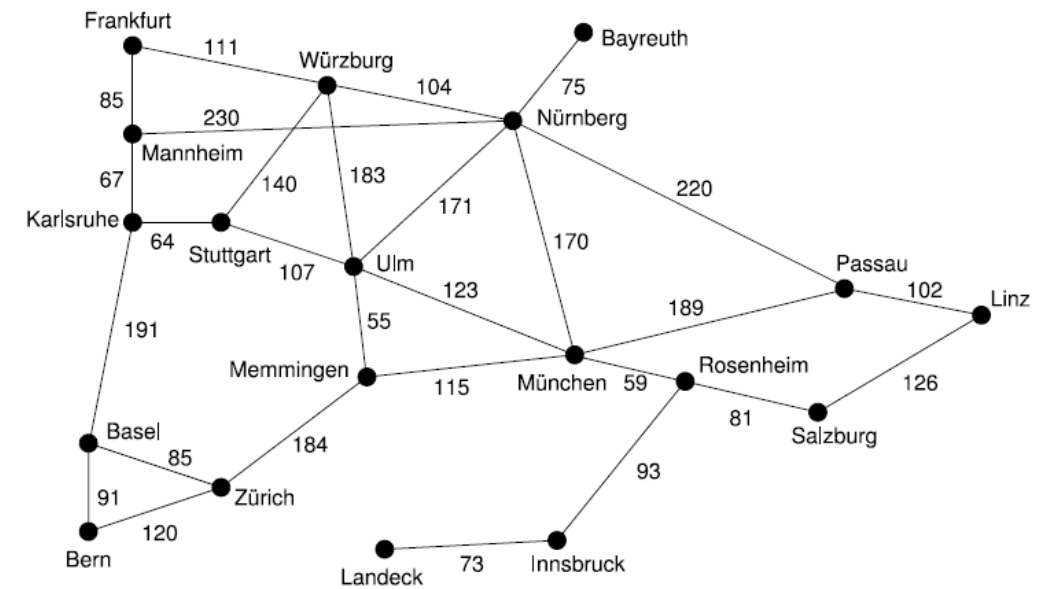


Figure 4.6: The graph of southern Germany as an example of a search task with a cost function

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction – example 2

- To find the route with **minimal length**, the costs must be taken into account because they are *not constant* as they were in the 8-puzzle.

Definition 3

*A search algorithm is called **optimal** if it, if a solution exists, always find the solution with the **lowest cost**.*

- The 8-puzzle problem is **deterministic**, which means that every action leads from a state to a unique successor state.
- It is furthermore **observable**, that is, the agent always knows which state it is in.
- *In route planning in real applications both characteristics are not always given.*
- The action “Drive from Munich to Ulm” may – for example because of an **accident** – lead to the successor state “Munich”. It can also occur that the traveler no longer knows where he is because he got lost. We want to ignore these kinds of complications at first.
- Therefore in this chapter we will only look at problems that are **deterministic** and **observable**.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

1. Introduction – example 2

- Problems like the 8-puzzle, which are **deterministic** and **observable**, make action planning relatively simple because, due to having an **abstract model**, it is possible to find action sequences for the solution of the problem without actually carrying out the actions in the real world.
- In the case of the 8-puzzle, it is not necessary to actually move the squares in the real world to find the solution.
- We can find **optimal solutions** with so-called **offline algorithms**.
- *One faces much different challenges when, for example, building **robots** that are supposed to play **soccer**.* Here there will **never** be an exact abstract model of the actions.
- For example, a robot that kicks the ball in a specific direction cannot predict with certainty where the ball will move because, among other things, it does not know whether an opponent will catch or deflect the ball.
- Here **online algorithms** are needed, which make decisions based on **sensor** signals in every situation.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

Contents

1. Introduction
- 2. Uninformed Search**
3. Heuristic Search
4. Games with opponents
5. Heuristic Evaluation Functions
6. State of the Art

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

2. Uninformed Search – Breadth-First Search (BFS)

- In **breadth-first search**, the search tree is explored from *top to bottom* according to the algorithm in **Figure 4.7** until a solution is found.

```
BREADTHFIRSTSEARCH(NodeList, Goal)
NewNodes = ∅
For all Node ∈ NodeList
    If GoalReached(Node, Goal)
        Return("Solution found", Node)
    NewNodes = Append(NewNodes, Successors(Node))
If NewNodes ≠ ∅
    Return(BREADTH-FIRST-SEARCH(NewNodes, Goal))
Else
    Return("No solution")
```

Figure 4.7: The algorithm for breadth-first search

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

2. Uninformed Search – Breadth-First Search

- First every node in the node list is tested for whether it is a **goal** node, and in the case of success, the program is *stopped*.
- Otherwise **all successors** of the node are generated.
- The search is then continued **recursively** on the list of all newly generated nodes.
- *The whole thing repeats until no more successors are generated.*

```
BREADTHFIRSTSEARCH(NodeList, Goal)
NewNodes = ∅
For all Node ∈ NodeList
    If GoalReached(Node, Goal)
        Return("Solution found", Node)
    NewNodes = Append(NewNodes, Successors(Node))
If NewNodes ≠ ∅
    Return(BREADTH-FIRST-SEARCH(NewNodes, Goal))
Else
    Return("No solution")
```

Figure 4.7: The algorithm for breadth-first search

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

2. Uninformed Search – Breadth-First Search

- The **Breadth-First Search (BFS)** algorithm is **generic**.
- That is, it works for arbitrary applications if the two application-specific functions “*GoalReached*” and “*Successors*” are provided.
- “*GoalReached*” calculates whether the argument is a goal node and “*Successors*” calculates the **first** of all **successor** nodes of its argument.
- **Figure 4.8** shows a snapshot of breadth-first search. The nodes are numbered according to the **order** they were generated. The successors of nodes 11 and 12 have not yet been generated.

```
BREADTHFIRSTSEARCH(NodeList, Goal)
NewNodes = ∅
For all Node ∈ NodeList
    If GoalReached(Node, Goal)
        Return(“Solution found”, Node)
    NewNodes = Append(NewNodes, Successors(Node))
If NewNodes ≠ ∅
    Return(BREADTH-FIRST-SEARCH(NewNodes, Goal))
Else
    Return(“No solution”)
```

Figure 4.7: The algorithm for breadth-first search

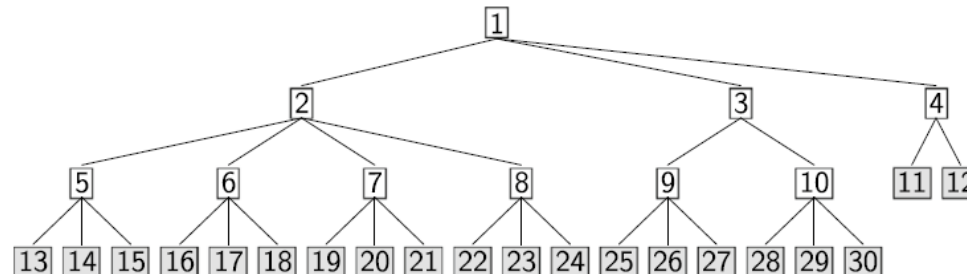


Figure 4.8: Breadth-first search during the expansion of the third-level nodes

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

2. Uninformed Search – Breadth-First Search

- **Analysis**
- Since **breadth-first search** *completely* searches through *every depth* and reaches *every depth* in **finite time**, it is **complete** if the branching factor b is **finite**.
- The **optimal** (that is, the shortest) solution is found if the costs of all actions are the **same**.
- **Computation time** and **memory space** grow **exponentially** with the **depth** of the tree.

```
BREADTHFIRSTSEARCH(NodeList, Goal)
NewNodes = ∅
For all Node ∈ NodeList
    If GoalReached(Node, Goal)
        Return("Solution found", Node)
    NewNodes = Append(NewNodes, Successors(Node))
If NewNodes ≠ ∅
    Return(BREADTH-FIRST-SEARCH(NewNodes, Goal))
Else
    Return("No solution")
```

Figure 4.7: The algorithm for breadth-first search

2. Uninformed Search – Breadth-First Search

- **Analysis**
- For a tree with **constant branching factor** b and **depth** d , the total **compute time** is thus given by

$$c \cdot \sum_{i=0}^d b^i = \frac{b^{d+1} - 1}{b - 1} = O(b^d)$$

- Although only the **last level** is saved in memory, the **memory space** is also $O(b^d)$.
- With the speed of today's computers, which can generate billions of nodes within minutes, main memory quickly fills up and the search ends.
- The problem of the **shortest solution**, *not always being found*, can be solved by the so-called **Uniform Cost Search**, in which the node with the **lowest cost** from the list of nodes (which is **sorted ascendingly by cost**) is always **expanded** and the new nodes **sorted in**. [URL: [Uniform Cost Search in AI](#)]
- Thus we find the **optimal** solution.

```
BREADTHFIRSTSEARCH(NodeList, Goal)
NewNodes = ∅
For all Node ∈ NodeList
    If GoalReached(Node, Goal)
        Return("Solution found", Node)
    NewNodes = Append(NewNodes, Successors(Node))
If NewNodes ≠ ∅
    Return(BREADTH-FIRST-SEARCH(NewNodes, Goal))
Else
    Return("No solution")
```

Figure 4.7: The algorithm for breadth-first search

2. Uninformed Search – Depth-First Search (DFS)

- In **depth-first search** only a *few nodes* are stored in memory at one time.
- After the expansion of a node only its **successors** are saved and the **first successor node** is immediately **expanded**.
- Thus the search quickly becomes *very deep*.
- Only when a node has **no successors** and the search fails at that depth, the next open node is expanded via **backtracking** on the **last branch**, and so on.
- URL: [Backtracking](#)

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

2. Uninformed Search – Depth-First Search

- We can best perceive this in the elegant recursive algorithm in **Figure 4.9** and in the search tree in **Figure 4.10**.
- All nodes at depth **three** are unsuccessful and cause *backtracking*. The nodes are numbered in the order they were generated.

```
DEPTHFIRSTSEARCH(Node, Goal)
```

```
  If GoalReached(Node, Goal) Return("Solution found")
```

```
  NewNodes = Successors(Node)
```

```
  While NewNodes  $\neq \emptyset$ 
```

```
    Result = DEPTH-FIRST-SEARCH(First(NewNodes), Goal)
```

```
    If Result = "Solution found" Return("Solution found")
```

```
    NewNodes = Rest(NewNodes)
```

```
  Return("No solution")
```

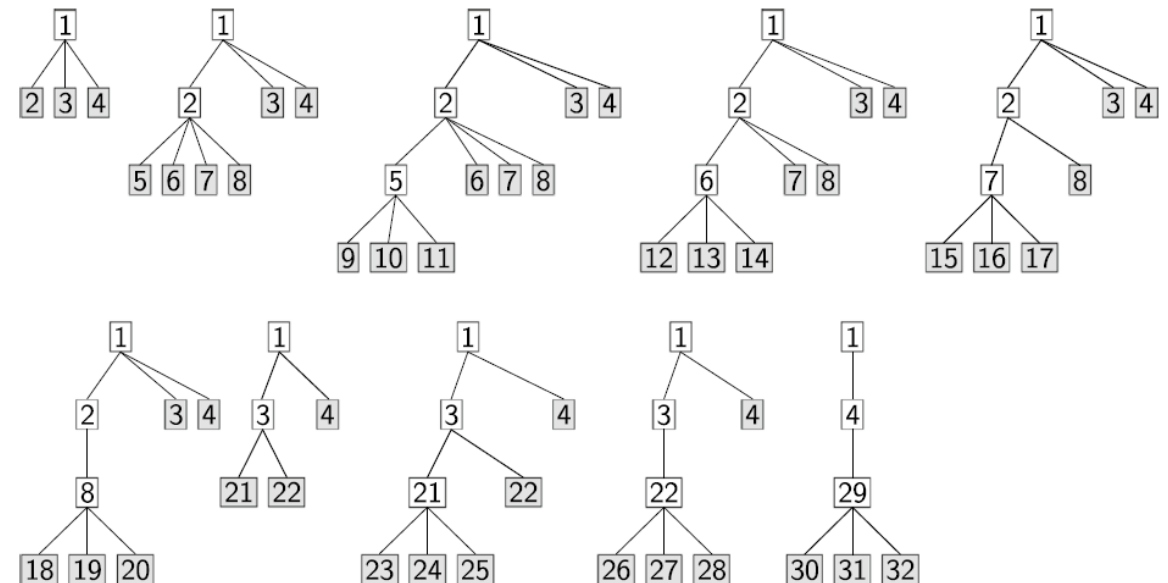


Figure 4.9: The algorithm for depth-first search

Figure 4.10: Execution of depth-first search

2. Uninformed Search – Depth-First Search

- Analysis
- **Depth-first search (DFS)** requires *much less memory* than **breadth-first search** because at most b nodes are saved at each depth.
- Thus we need at most $b \cdot d$ memory cells.
- However, **depth-first search** is *not complete* for **infinitely** deep trees because depth-first search runs into an **infinite loop** when there is *no solution* in the **far left branch**.
- Therefore the question of finding the optimal solution is obsolete.
- *Because of the infinite loop, no bound on the computation time can be given.*
- In the case of a **finitely** deep search tree with depth d , a total of b^d nodes are generated.
- Thus the computation time grows, just as in breadth-first search, **exponentially** with **depth**.
- We can make the search tree **finite** by setting a **depth limit**.
- Now if no solution is found in the pruned search tree, there can nonetheless be solutions outside the limit. Thus the search becomes **incomplete**. There are obvious ideas, however, for getting the search to **completeness**.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

2. Uninformed Search – Iterative Deepening

- We begin the **depth-first search** with a depth limit of 1.
- If no solution is found, we raise the limit by 1 and start searching from the beginning, and so on, as shown in **Figure 4.11**.
- **Figure 4.11** illustrates the schematic representation of the development of the search tree in **iterative deepening** with limits from 1 to 7. The *breadth* of the tree corresponds to a **branching factor** of 2.

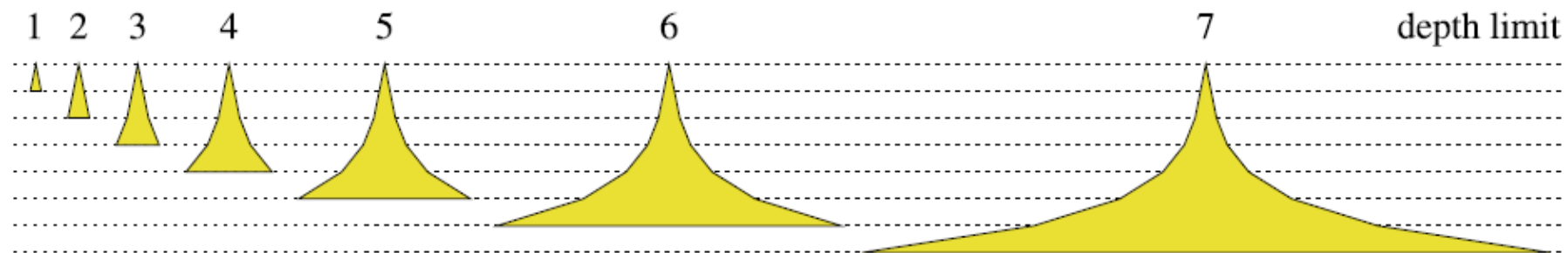


Figure 4.11: Iterative Deepening

2. Uninformed Search – Iterative Deepening

- We must augment the **depth-first search** program with the additional parameters “*Depth*” and “*Limit*”.
- “*Depth*” is raised by one at the recursive call and the head line of the while loop is replaced by “**While** *NewNodes* $\neq$ 0 **and** *Depth* < *Limit*”.
- The modified algorithm is represented in **Figure 4.12**.
- The algorithm for **iterative deepening** is illustrated in **Figure 4.12**, which calls the slightly modified **depth-first search** with a **depth limit**.

```
ITERATIVEDEEPENING(Node, Goal)
DepthLimit = 0
Repeat
    Result = DEPTHFIRSTSEARCH-B(Node, Goal, 0, DepthLimit)
    DepthLimit = DepthLimit + 1
Until Result = “Solution found”
```

```
DEPTHFIRSTSEARCH-B(Node, Goal, Depth, Limit)
If GoalReached(Node, Goal) Return(“Solution found”)
NewNodes = Successors(Node)
While NewNodes  $\neq$   $\emptyset$  And Depth < Limit
    Result =
        DEPTHFIRSTSEARCH-B(First(NewNodes), Goal, Depth + 1, Limit)
    If Result = “Solution found” Return(“Solution found”)
    NewNodes = Rest(NewNodes)
Return(“No solution”)
```

Figure 4.12: The algorithm for iterative deepening

2. Uninformed Search – Iterative Deepening

- [Analysis](#)
- The memory requirement is the same as in **depth-first search**.
- One could argue that repeatedly re-starting **depth-first search** at depth zero causes a lot of redundant work. For large branching factors this is not the case.
- *We now show that the sum of the number of nodes of all depths up to the **one before last** ($d_{max} - 1$) in all trees searched is much smaller than the number of nodes in the **last tree** searched.*
- Let $N_b(d)$ be the number of nodes of a search tree with **branching factor** b and **depth** d and d_{max} be the last depth searched. The last tree searched contains

$$N_b(d_{max}) = \sum_{i=0}^{d_{max}} b^i = \frac{b^{d_{max}+1}-1}{b-1} \text{ nodes.}$$

2. Uninformed Search – Iterative Deepening

- [Analysis](#)
- All tree searched beforehand together have

$$\begin{aligned}\sum_{d=1}^{d_{max}-1} N_b(d) &= \sum_{d=1}^{d_{max}-1} \frac{b^{d+1} - 1}{b - 1} = \frac{1}{b - 1} \left(\left(\sum_{d=1}^{d_{max}-1} b^{d+1} \right) - d_{max} + 1 \right) \\ &= \frac{1}{b - 1} \left(\left(\sum_{d=2}^{d_{max}} b^d \right) - d_{max} + 1 \right) \\ &= \frac{1}{b - 1} \left(\frac{b^{d_{max}+1} - 1}{b - 1} - 1 - b - d_{max} + 1 \right) \\ &\approx \frac{1}{b-1} \left(\frac{b^{d_{max}+1}-1}{b-1} \right) = \frac{1}{b-1} N_b(d_{max}) \text{ nodes}\end{aligned}$$

- For $b > 2$ this is *less* than the number $N_b(d_{max})$ of nodes in the **last tree**. For $b = 20$ the **first** $d_{max} - 1$ trees together contain only about $\frac{1}{b-1} = \frac{1}{19}$ of the number of nodes in the **last tree**.

2. Uninformed Search – Comparison

- The described search algorithms have been put side-by-side in **Table 4.1**.
- **Table 4.1** illustrates a comparison of the **uninformed search algorithms**. (*) means that the statement is only true given a **constant action cost**.

	Breadth-first search	Uniform cost search	Depth-first search	Iterative deepening
Completeness	Yes	Yes	No	Yes
Optimal solution	Yes (*)	Yes	No	Yes (*)
Computation time	b^d	b^d	∞ or b^d	b^d
Memory use	b^d	b^d	bd	bd

Table 4.1: Comparison of the uninformed search algorithms

2. Uninformed Search – Comparison

	Breadth-first search	Uniform cost search	Depth-first search	Iterative deepening
Completeness	Yes	Yes	No	Yes
Optimal solution	Yes (*)	Yes	No	Yes (*)
Computation time	b^d	b^d	∞ or b^d	b^d
Memory use	b^d	b^d	bd	bd

Table 4.1: Comparison of the uninformed search algorithms

- We can clearly see that **iterative deepening** is the winner of this test because it gets the best grade in all categories.
- In fact, of all four algorithms presented, it is the only practically usable one.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

2. Uninformed Search – Comparison

	Breadth-first search	Uniform cost search	Depth-first search	Iterative deepening
Completeness	Yes	Yes	No	Yes
Optimal solution	Yes (*)	Yes	No	Yes (*)
Computation time	b^d	b^d	∞ or b^d	b^d
Memory use	b^d	b^d	bd	bd

Table 4.1: Comparison of the uninformed search algorithms

- *We do indeed have a winner for this test, although for **realistic applications** it is usually not successful.*
- For non-trivial inference systems the state space is many orders of magnitude **bigger**. All the computing power in the world will not help much more.
- Even for the 15-puzzle, the 8-puzzle's big brother, there are about $2 \cdot 10^{13}$ different states.
- Instead what is needed is an **intelligent search** that explores a **tiny fraction** of the search space and finds a solution there.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

2. Uninformed Search – Cycle Check

- *Nodes may be repeatedly visited during a search.*
- In the 8-puzzle, for example, every move can be immediately undone, which leads to **unnecessary cycles** of length two.
- Such cycles can be **prevented** by recording within each node all of its **predecessors** and, when expanding a node, comparing the newly created successor nodes with the predecessor nodes.
- This simple check costs only a small constant factor of additional memory space and increases the constant computation time c by an additional constant δ for the check itself for a total of $c + \delta$.
- This overhead for the cycle check is (hopefully) offset by a reduction in the cost of the search.
- The reduction depends, of course, on the particular application and therefore cannot be given in general terms.

2. Uninformed Search – Cycle Check

- For the 8-puzzle we obtain the result as follows.
- If, for example, during **breadth-first search** with effective branching factor b on a **finite** tree of depth d , the computation time *without* the cycle check is $c \cdot b^d$, the required time with the cycle check becomes

$$(c + \delta) \cdot (b - 1)^d$$

- The check thus practically always results in a clear gain because reducing the **branching factor** by **one** has an *exponentially* growing effect as the depth increases, whereas the additional computation time δ only somewhat *increases* the constant factor.
- *Now the question arises as to how a check on cycles of arbitrary length would affect the search performance.*
- The list of all **predecessors** must now be stored for each node, which can be done very efficiently.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

2. Uninformed Search – Cycle Check

- During the search, each newly created node must now be compared with all its **predecessors**.
- The computation time of **depth-first search** or **breadth-first search** is given by

$$c_1 \cdot \sum_{i=0}^d b^i + c_2 \cdot \sum_{i=0}^d i \cdot b^i$$

- Here, the **first term** is the already-know **cost of generating the nodes** and the **second term** is the cost of the **cycle check**.
- We can show that for large values of b and d

$$\sum_{i=0}^d i \cdot b^i \approx d \cdot b^d$$

- *The complexity of the search with the full cycle check therefore increases by a factor of d faster than for the search without a cycle check.*
- In search trees that are not very deep, this extra complexity is not important.
- In summary, we can conclude that the **cycle check** implies hardly any additional overhead and is therefore worthwhile for applications with repeatedly occurring nodes.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

Contents

1. Introduction
2. Uninformed Search
- 3. Heuristic Search**
4. Games with opponents
5. Heuristic Evaluation Functions
6. State of the Art

3. Heuristic Search

- **Heuristics** are problem-solving strategies which in many cases find a solution **faster** than **uninformed search**.
- *However, this is not guaranteed! Heuristic search could require a lot more time and can even result in the solution not being found.*
- Heuristic decisions are closely linked with the need to make **real-time decisions with limited resources**.
- *In practice a good solution found quickly is **preferred** over a solution that is **optimal**, but very expensive to derive.*
- A heuristic evaluation function $f(s)$ for states is used to mathematically model a **heuristic**.
- The goal is to find, with little effort, a solution to the stated search problem with **minimal total cost**.
- Please note that there is a subtle difference between the *effort* to find a solution and the *total cost* of this solution.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

3. Heuristic Search

- Next we will modify the **breadth-first search** algorithm by adding the **evaluation function** to it.
- The currently open nodes are no longer expanded left to right by row, but rather according to their **heuristic rating**.
- *From the set of open nodes, the node with the **minimal rating** is always expanded **first**.*
- This is achieved by *immediately* evaluating nodes as they are expanded and **sorting** them into the list of open nodes.
- The list may then contain nodes from *different* depths in the tree.
- Because heuristic evaluation of states is very important for the search, we will differentiate from now on between states and their associated nodes.
- The **node** contains the **state** and further information relevant to the search, such as its **depth** in the search tree and the **heuristic rating** of the state.
- As a result, the function “*Successors*”, which generates the successors (children) of a node, must also *immediately* calculate for these successor nodes their **heuristic ratings** as a component of each node.

3. Heuristic Search

- We define the general search algorithm **HeuristicSearch** in **Figure 4.13**.
- The node list is initialized with the **starting** nodes.
- Then, in the loop, the **first node** from the list is removed and tested for whether it is a solution node.
- If not, it will be expanded with the function **“Successors”** and its **successors** added to the list with the function **“SortIn”**.
- **“SortIn(X,Y)”** inserts the elements from the **unsorted** list X into **ascendingly sorted** list Y.
- The **heuristic rating** is used as the **sorting key**.
- Thus it is guaranteed that the **best node** (that is, the one with the *lowest heuristic value*) is always at the **beginning** of the list.

```
HEURISTICSEARCH(Start, Goal)
NodeList = [Start]
While True
    If NodeList = ∅ Return(“No solution”)
    Node = First(NodeList)
    NodeList = Rest(NodeList)
    If GoalReached(Node, Goal) Return(“Solution found”, Node)
    NodeList = SortIn(Successors(Node),NodeList)
```

Figure 4.13: The algorithm for heuristic search

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

3. Heuristic Search

- **Depth-first** and **breadth-first** search also happen to be **special cases** of the function **HeuristicSearch**. We can easily generate them by plugging in the appropriate **evaluation function**.
- *The best heuristic would be a function that calculates the actual costs from **each node** to the **goal**.*
- To do that, however, would require a traversal of the entire search space, which is exactly what the heuristic is supposed to prevent.
- Therefore, we need a heuristic that is **fast** and **simple** to compute.
- *How do we find such a heuristic?*
- An interesting idea for finding a heuristic is a *simplification* of the problem.
- The original task is **simplified** enough that it can be solved with little computational time.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

3. Heuristic Search

- The costs from a *state* to the *goal* in the *simplified* problem than serve as an **estimate** for the actual problem (see **Figure 4.14**).
- This **cost estimate** we denote *h*.

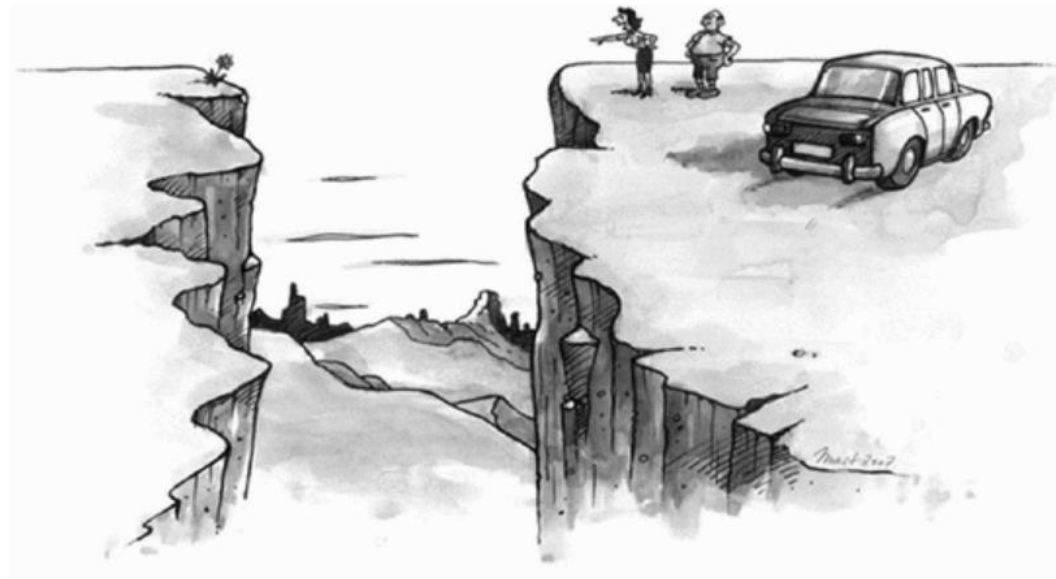


Figure 4.14: He: “Dear, think of the fuel cost! I’ll pluck one for you somewhere else.”
She: “No, I want that one over there!”

3. Heuristic Search – Greedy Search

- It seems sensible to choose the state with the **lowest estimated h value** (that is, the one with the *lowest estimated cost*) from the list of currently available states.
- The **cost estimate** then can be used directly as the **evaluation function**.
- For the evaluation in the function **HeuristicSearch** we set $f(s) = h(s)$.
- This can be seen clearly in the trip planning **example 2**.
- We set up the task of finding the straight line path from city to city (that is, the **flying distance**) as a simplification of the problem.

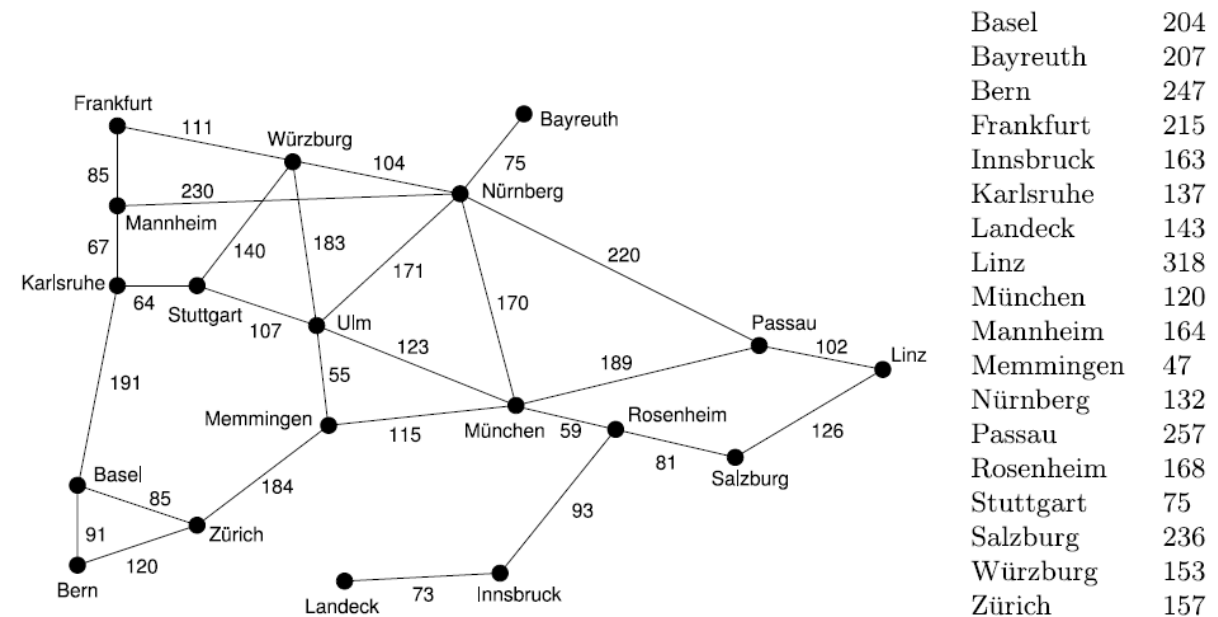


Figure 4.15: City graph with flying distances from all cities to Ulm

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

3. Heuristic Search – Greedy Search

- Instead of searching the optimal route, we first determine from every node a route with *minimal flying distance* to the **goal**.
- We choose **Ulm** as the **destination**.
- Thus the **cost estimate function** becomes
$$h(s) = \text{flying distance from city } s \text{ to Ulm}$$
- The **flying distances** from all cities to **Ulm** are given in **Figure 4.15**, next to the graph.

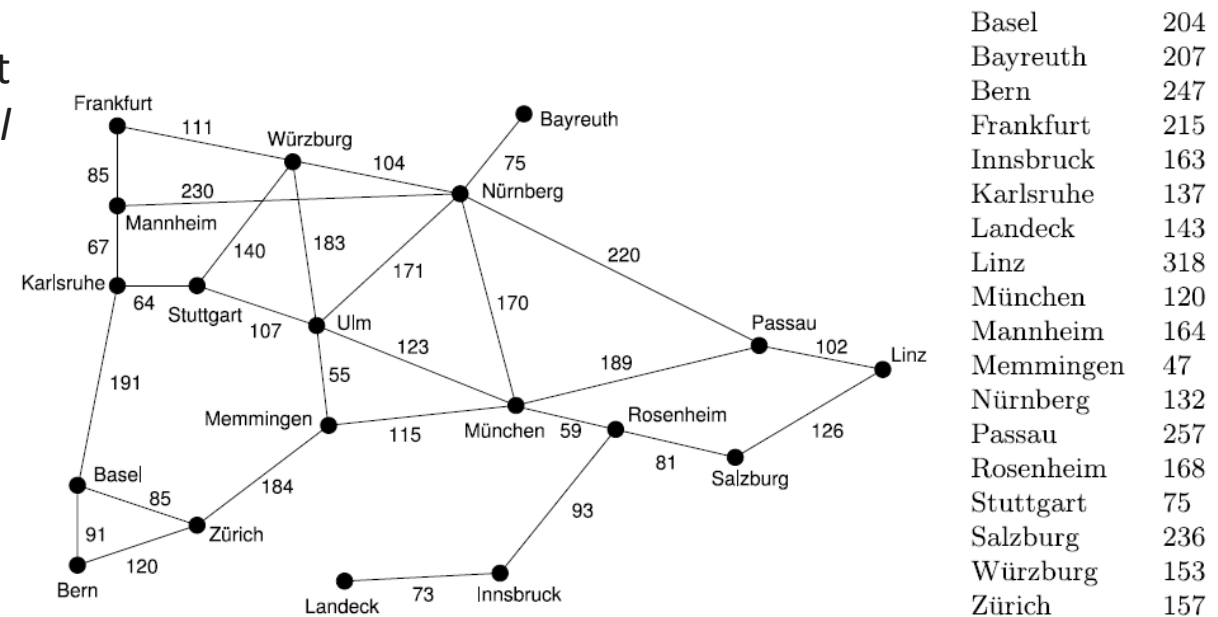


Figure 4.15: City graph with flying distances from all cities to Ulm

3. Heuristic Search – Greedy Search

- The search tree for **starting** in **Linz** is represented in **Figure 4.16**.
- **Figure 4.16** illustrates **Greedy Search**, from **Linz** to **Ulm** (left) and from **Mannheim** to **Ulm** (right).
- The node list data structure for the **left** search tree, sorted by the node rating *before* the expansion of the node *München* is given.
- We can see that the tree is very slender. The search thus finishes quickly.

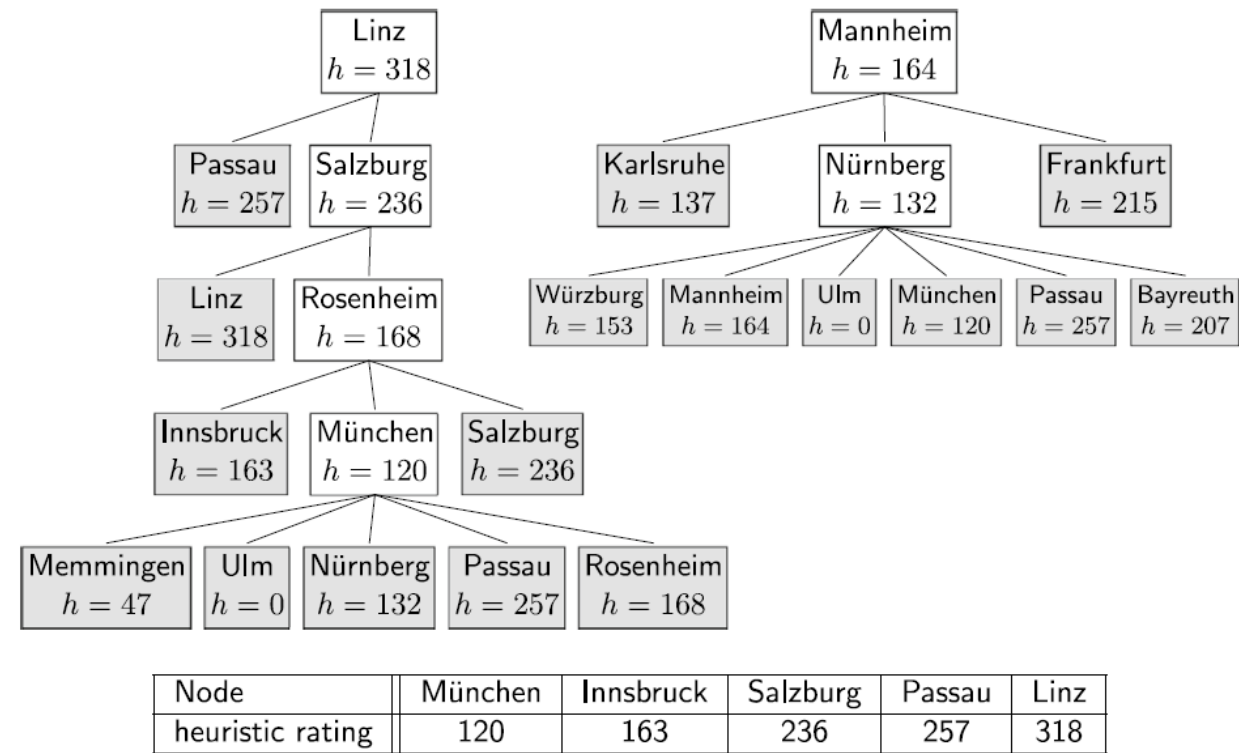


Figure 4.16: Greedy Search

3. Heuristic Search – Greedy Search

- Unfortunately, this search does not always find the optimal solution.
- For example, this algorithm fails to find the optimal solution when starting in **Mannheim** (Figure 4.16 – right).
- The **Mannheim – Nürnberg – Ulm** path has a length of **401 km**. The route **Mannheim-Karlsruhe-Stuttgart-Ulm** would be significantly shorter at **238 km**.

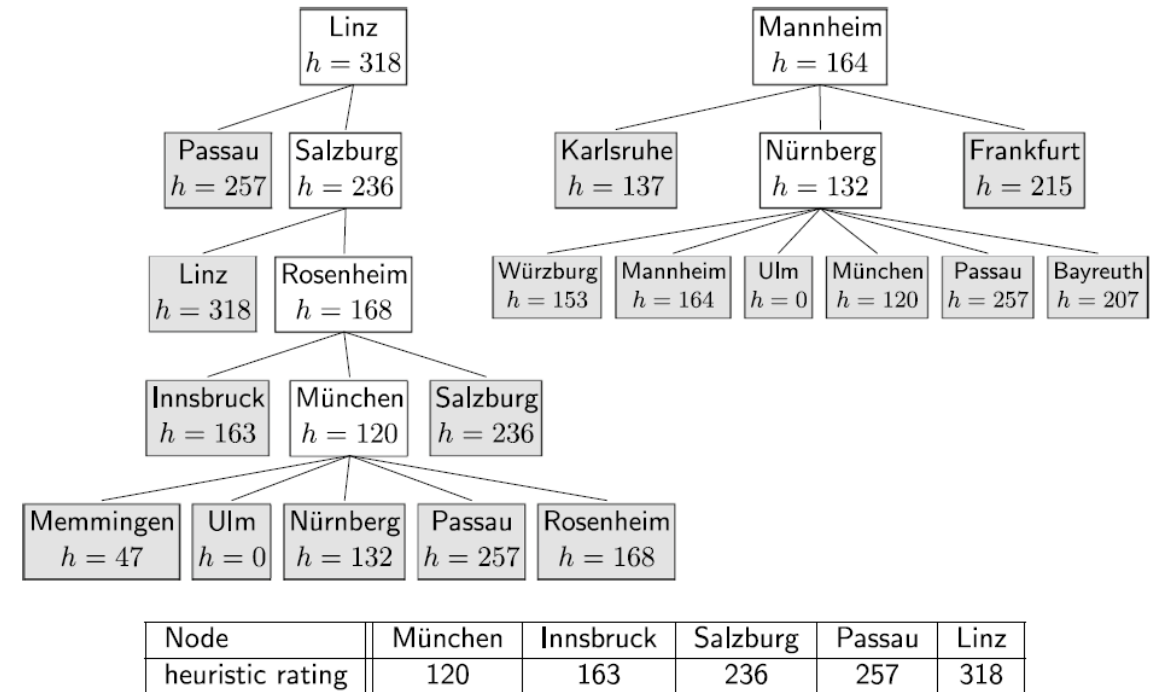


Figure 4.16: Greedy Search

- As we observe the graph, the cause of this problem becomes clear.
- Nürnberg (132)** is in fact somewhat closer than *Karlsruhe* (137) to *Ulm*, but the distance from *Mannheim* to *Nürnberg* is significantly greater than that from *Mannheim* to *Karlsruhe*.
- The heuristic only looks ahead “**greedily**” to the goal instead of also taken into account the stretch that has already been laid down to the current node.
- This is why we give it the name **greedy search**.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

3. Heuristic Search – A*-Search

- We now want to take into account the costs that have accrued during the search up to the current node s .

- First we define the **cost function**

$g(s) = \text{Sum of accrued costs from the start to the current node}$

- then **add** to that the *estimated cost* to the goal and obtain as the **heuristic evaluation function**

$$f(s) = g(s) + h(s)$$

- Now we add yet another small, but important requirement

Definition 4 *A heuristic cost estimate function $h(s)$ that never overestimates the actual cost from state s to the goal is called **admissible**.*

- The function **HeuristicSearch** together with an **evaluation function** $f(s) = g(s) + h(s)$ and an **admissible heuristic function** h is called **A*-algorithm**.
- This famous algorithm is **complete** and **optimal**. A* thus always finds the **shortest solution** for every solvable search problem.

3. Heuristic Search – A*-Search

- First we apply the **A*-algorithm** to the example. We are looking for the shortest path from *Frankfurt* to *Ulm*.
- In the top part of **Figure 4.17** we see that the successors of *Mannheim* are generated before the successors of *Würzburg*.
- **Figure 4.17** illustrates two snapshots of the A* search tree for the optimal route from *Frankfurt* to *Ulm*. In the boxes below the name of the city s we show $g(s)$, $h(s)$, $f(s)$. Numbers in parentheses after the city names show the order in which the nodes have been generated by the “*Successor*” function.
- The **optimal** solution *Frankfurt-Würzburg-Ulm* is generated shortly thereafter in the **eight** step, but it is not yet recognized as such.

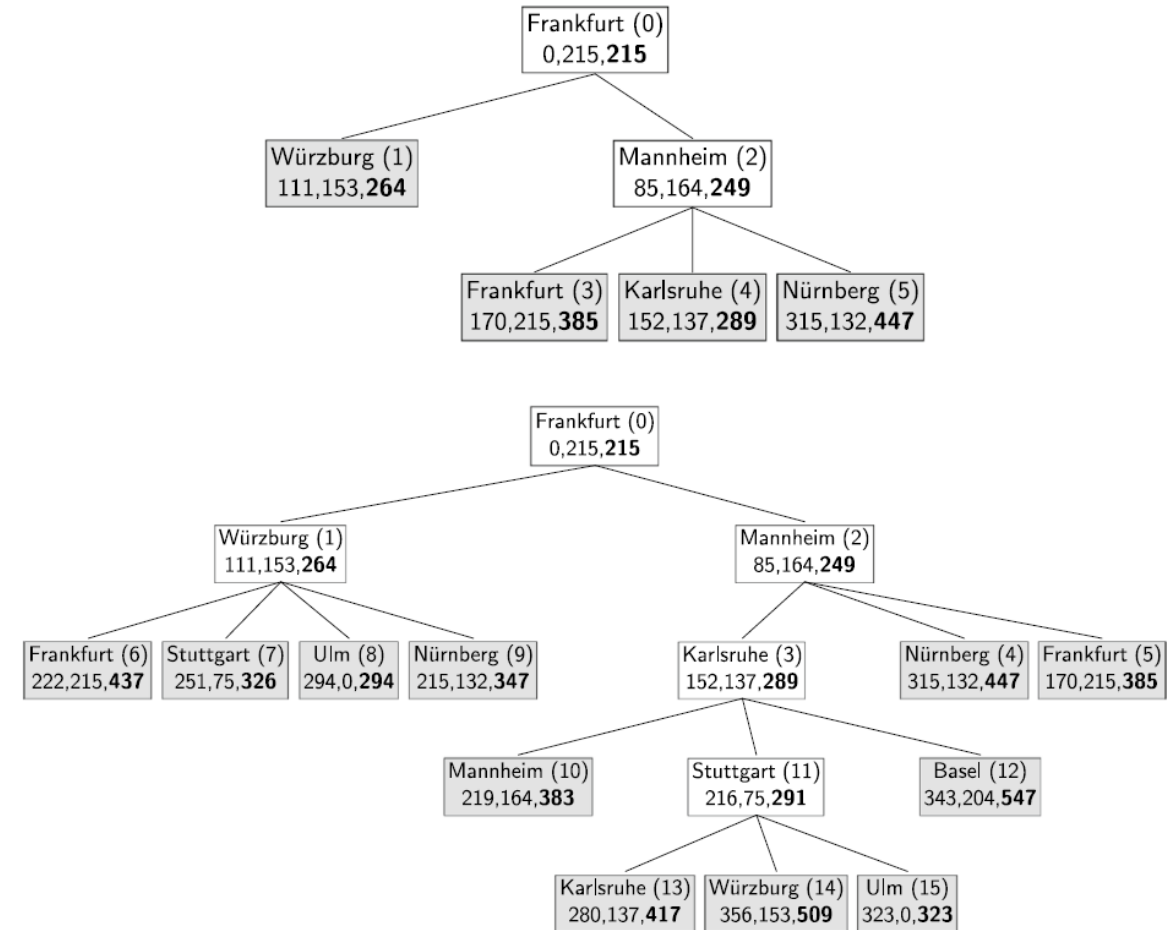


Figure 4.17: Two snapshots of the A* search tree for the optimal route from Frankfurt to Ulm

3. Heuristic Search – A*-Search

- Thus the algorithm does not terminate yet because the node *Karlsruhe* (3) has a better (lower) f value (289) and this is ahead of the node *Ulm* (8) in line (294).
- Only when all f values [Karlsruhe (417), Würzburg (509), Ulm (323)] are greater than or equal to that of the solution node *Ulm* (8) (294) have we ensured that we have an **optimal** solution.
- Otherwise there could potentially be another solution with lower costs.
- We will now show that this is true generally.

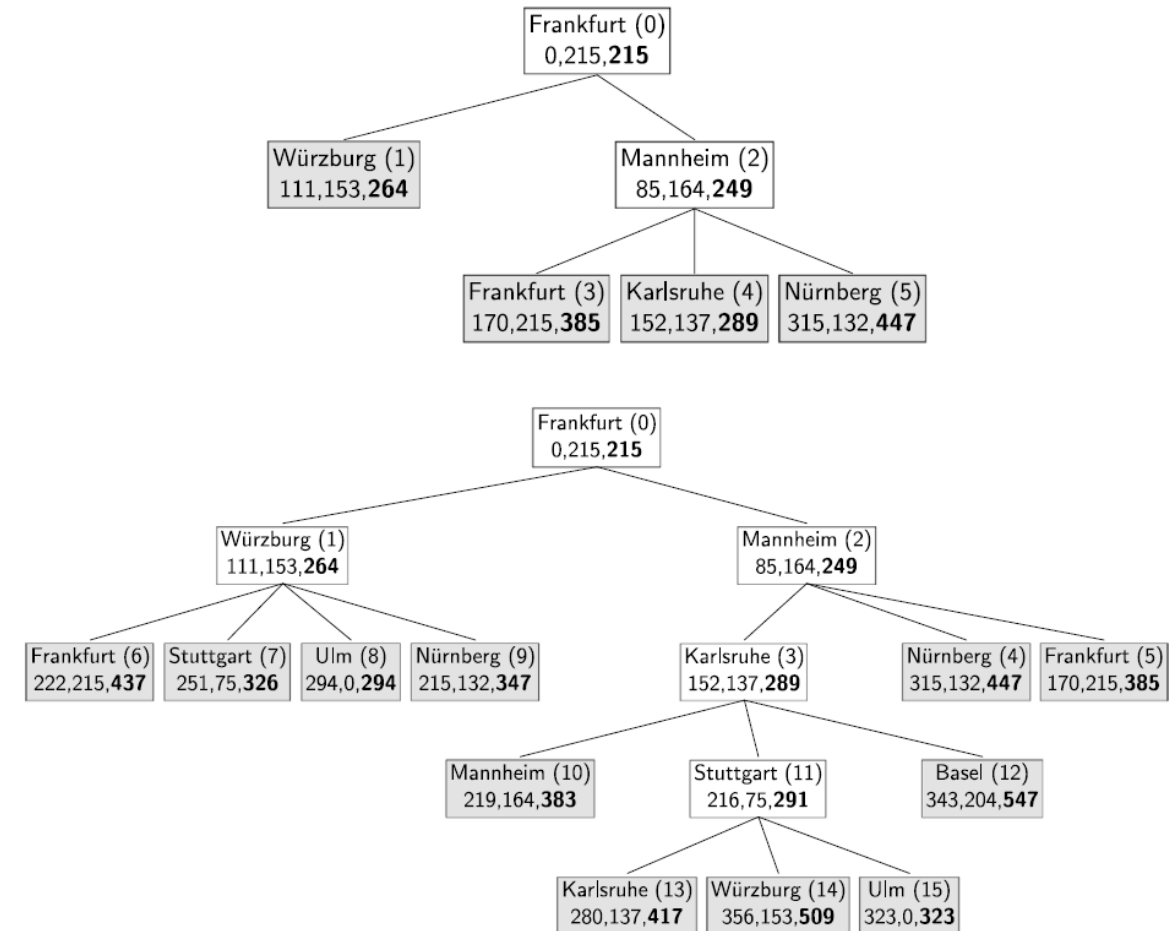


Figure 4.17: Two snapshots of the A* search search tree for the optimal route form Frankfurt to Ulm

3. Heuristic Search – A*-Search

Theorem 2

The A* algorithm is **optimal**. That is, it always finds the solution with the **lowest total cost** if the heuristic h is **admissible**.

- **Proof**
- In the **HeuristicSearch** algorithm, every newly generated node s is **sorted** in by the function “**SortIn**” according to its heuristic rating $f(s)$.
- The node with the **smallest rating value** thus is at the **beginning** of the list.
- If the node l at the beginning of the list is a **solution** node, then no other node has a better heuristic rating.

AI

Ch

AI

Ch

- **Proof**

- For all other nodes s it is true that $f(l) \leq f(s)$. Because the heuristic is **admissible**, no better solution l' can be found, even after expansion of all other nodes (see **Figure 4.18**).

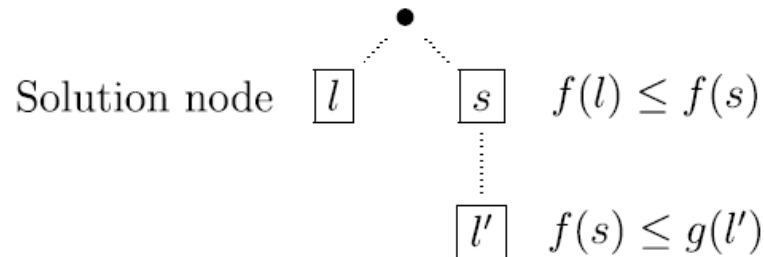


Figure 4.18: The first solution node I found by A never has a higher cost than another arbitrary node I'*

- Written formally

$$g(l) + 0 = g(l) + h(l) = f(l) \leq f(s) = g(s) + h(s) \leq g(l')$$
- The **first** equality holds because l is a **solution node** with $h(l) = 0$. The **second** is the definition of f . The **third** (in)equality holds because the list of open nodes is **sorted** in *ascending order*.
- The **fourth** equality is again the definition of f . Finally, the last (in)equality is the admissibility of the heuristic, which **never overestimates** the cost from node s to an arbitrary solution.
- Thus it is shown that $g(l) \leq g(l')$, that is, that the discovered solution l is **optimal**.

3. Heuristic Search – Route Planning with the A* Search Algorithm

- Many current **car navigation systems** use the **A*** algorithm.
- The simplest, but very good **heuristic** for computing A* is the **straight-line distance** from the current node to the destination.

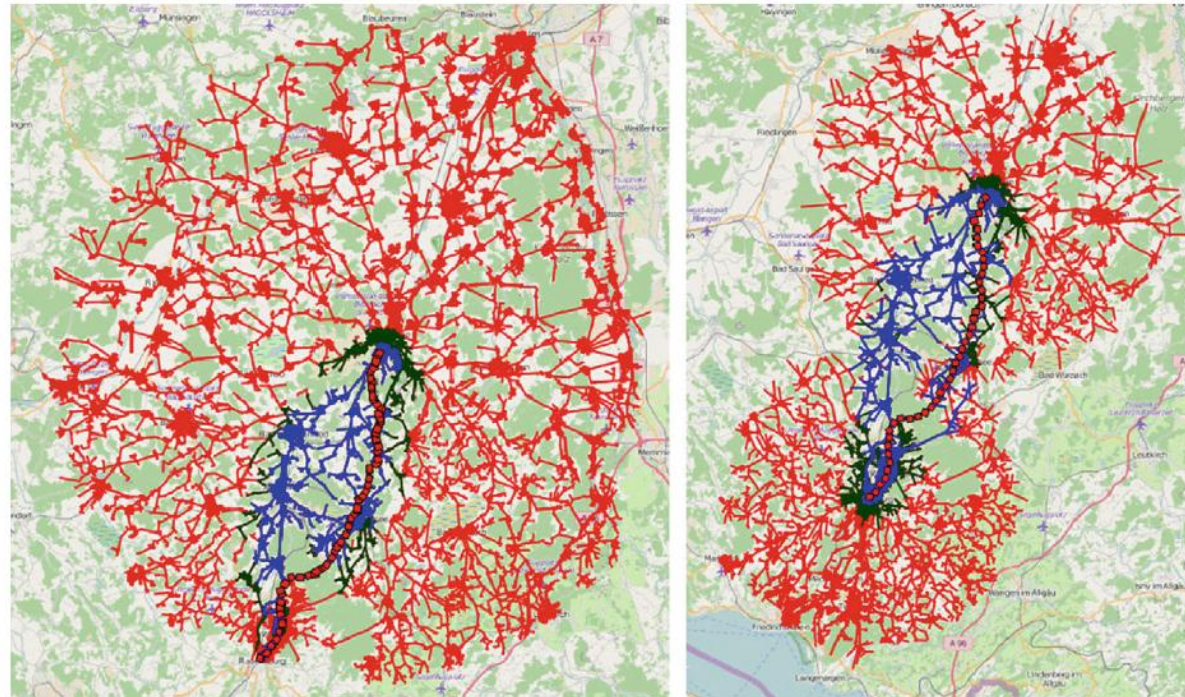
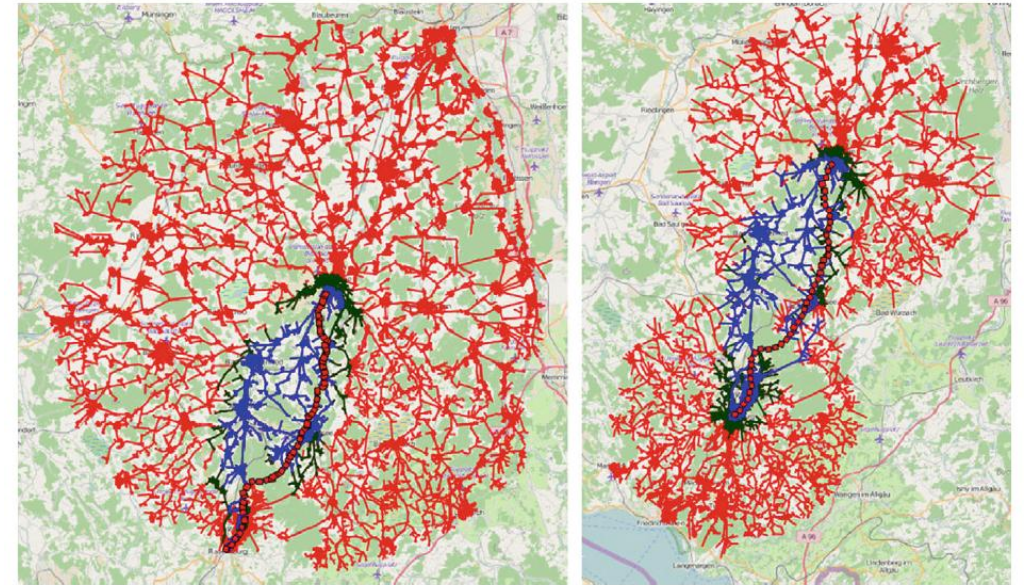


Figure 4.19: A* search tree

3. Heuristic Search

Route Planning with the A* Search Algorithm

- **Figure 4.19** illustrates the A* search tree **without** heuristic (red), with straight-line distance (dark green) and with landmarks (blue).
- The **left** image shows **unidirectional** search and the **right** shows **bidirectional** search. Note that the **green** edges are covered by **blue** and the **red** edges are covered by **green** and **blue**.
- The use of 5 to 60 so-called landmarks is somewhat better.
- For these *randomly* chosen points the shortest paths to and from all nodes on the map are calculated in a precomputation step.



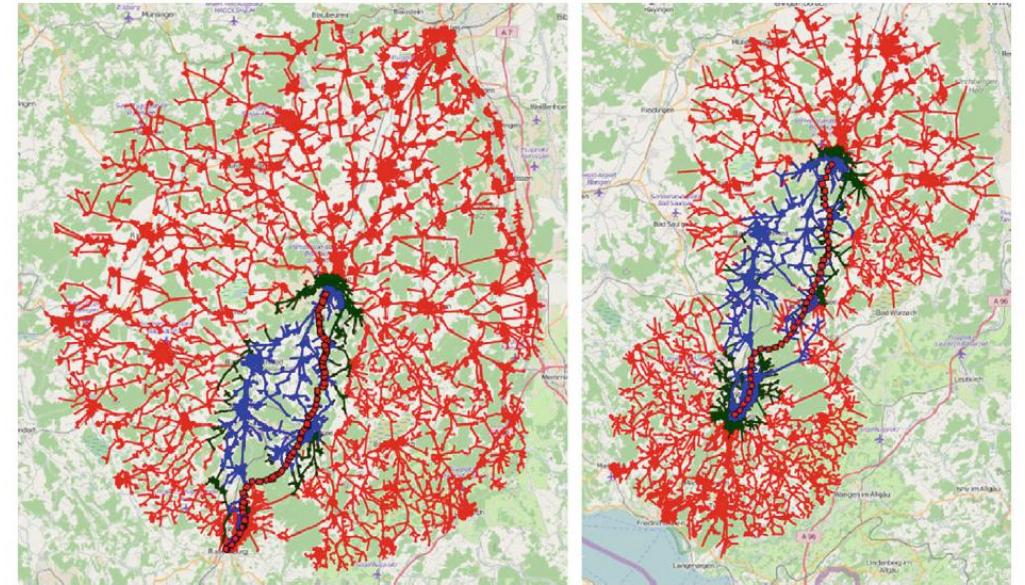
3. Heuristic Search

Route Planning with the A* Search Algorithm

- Let l be such a landmark, s , the current node, and z the destination node.
- Also let $c^*(x, y)$ be the **cost** of the **shortest path** from x to y .
- Then we obtain for the shortest path from s to l **the triangle inequality**

$$c^*(s, l) \leq c^*(s, z) + c^*(z, l)$$

- Solving for $c^*(s, z)$ results in the **admissible heuristic**
$$h(s) = c^*(s, l) - c^*(z, l) \leq c^*(s, z)$$
- This **heuristic** is better than the **straight-line distance** for route planning.

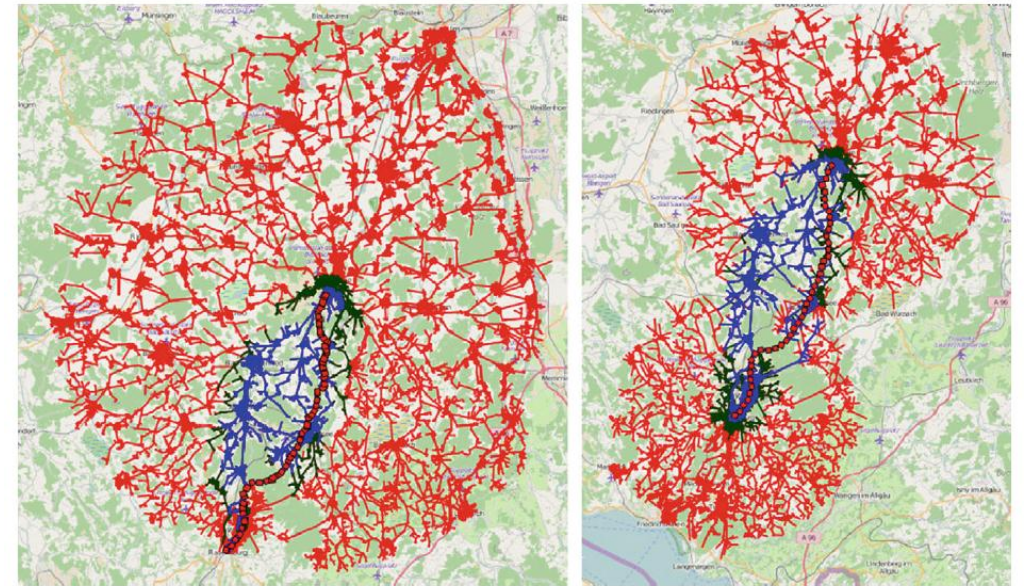


- On the other hand, it can be calculated **faster** than the straight-line distance.
- Due to precomputation, distances to the landmarks can be quickly retrieved from an **array**, whereas the Euclidean distances must be computed individually.

3. Heuristic Search

Route Planning with the A* Search Algorithm

- *It turns out that the landmark heuristic shrinks the search space even more.*
- This can be seen in the **left** image of **Figure 4.19**, which illustrates the search tree of A* search for planning a route from *Ravensburg* to *Biberach* (two towns in Southern Germany).
- The edges *without* any heuristic (i.e. with $h(s) = 0$) are plotted in **red** colour, **dark green** lines show the search tree using the **straight-line distance** heuristic, and the edges of the **landmark heuristic** with twenty landmarks are plotted in **blue**.
- The right image shows the same route using **bidirectional** search, where a route from *Ravensburg* to *Biberach* and one in the **opposite** direction are planned effectively in parallel.
- If the routes **meet**, given certain conditions of the heuristic, **an optimal route has been found**.



3. Heuristic Search – Route Planning with the A* Search Algorithm

- A quantitative analysis of the search tree sizes and the computation time on a PC can be found in **Table 4.2**.
- ***Table 4.2** illustrates the comparison of search tree size and computation time for route planning with and without each of the two heuristics.*
- The **landmark heuristic** is the clear winner.

	Unidirectional		Bidirectional	
	Tree size [nodes]	Comp. time [msec.]	Tree size [nodes]	Comp. time [msec.]
No heuristic	62000	192	41850	122
Straight-line distance	9380	86	12193	84
Landmark heuristic	5260	16	7290	16

Table 4.2: Comparison of search tree size and computation time for route planning

3. Heuristic Search – Route Planning with the A* Search Algorithm

- Observing *unidirectional* search, we see that both heuristics clearly reduce the search space.
- The computation times are truly interesting.
- In the case of the *landmark* heuristic, we see the computation time and the size of the search space reduced by a factor of about **12** (62000/5260).
- The cost of computing the heuristic is thus insignificant.

	Unidirectional		Bidirectional	
	Tree size [nodes]	Comp. time [msec.]	Tree size [nodes]	Comp. time [msec.]
No heuristic	62000	192	41850	122
Straight-line distance	9380	86	12193	84
Landmark heuristic	5260	16	7290	16

Table 4.2: Comparison of search tree size and computation time for route planning

3. Heuristic Search – Route Planning with the A* Search Algorithm

- The **straight-line distance**, however, results in a search space reduction of a factor **6.6** (62000/9380), but only an improvement of **2.2** (192/86) in run time due to the overhead of computing the Euclidean distance.
- In the case of the *bidirectional* search, in contrast to *unidirectional* search, we see a **significant reduction of the search space** even without heuristic.
- On the other hand, the search space is larger than the unidirection case for both heuristics.
- However, because the nodes are partitioned into **two sorted lists** in *bidirectional* search, the lists are handled faster and the resulting computation times are roughly the same.

	Unidirectional		Bidirectional	
	Tree size [nodes]	Comp. time [msec.]	Tree size [nodes]	Comp. time [msec.]
No heuristic	62000	192	41850	122
Straight-line distance	9380	86	12193	84
Landmark heuristic	5260	16	7290	16

Table 4.2: Comparison of search tree size and computation time for route planning

3. Heuristic Search – Route Planning with the A* Search Algorithm

- When planning a route, usually the driver cares more about **driving time** than the **distance** driven.
- We should thus adjust the heuristic accordingly and replace **straight-line distance** $d(s, z)$ with **time**
$$t(s, z) = \frac{d(s, z)}{v_{max}}.$$
- Here we have to divide by the **maximum average velocity**, which degrades the heuristic because it causes the heuristically estimated times to be much too small.
- The **landmark heuristic**, in contrast, builds on precomputed optimal routes and therefore does **not** degrade.
- *Thus, the search for a time-optimized route using landmark heuristic is significantly faster than with the modified straight-line distance.*
- The **contraction hierarchies** algorithm performs even better than A* with **landmark heuristic**.
- It is based on the idea of combining, in a precomputation step, several edges into so-called **shortcuts**, which are then used to reduce the search space.
- URL: [Contraction hierarchies algorithm](#)

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

3. Heuristic Search – IDA* Search

- The **A* search** inherits a quirk from **breadth-first search**.
- It has so many nodes in memory, which can lead to **very high memory use**.
- Furthermore, the list of open nodes must be **sorted**.
- Thus insertion of nodes into the list and removal of nodes from the list can no longer run in constant time, which increases the algorithm's complexity slightly.
- Based on the **heapsort algorithm**, we can structure the node list as a **heap with logarithmic time complexity** for insertion and removal of nodes.
- Both problems can be solved – similarly to breadth-first search – by **iterative deepening**.
- We work with **depth-first search** and **successively raise the limit**.
- However, rather than working with a depth limit, here we use a **limit for heuristic evaluation $f(s)$** .
- This process is called the **IDA* algorithm**.

- URL: [Heapsort algorithm](#)
- URL: [Iterative deepening A*](#)

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

3. Heuristic Search – Empirical Comparison of the Search Algorithms

- In **A***, or (alternatively) **IDA***, we have a search algorithm with many good properties.
- It is **complete** and **optimal**. It can thus be used without risk.
- The most important thing, however, is that it works with **heuristics**, and therefore can significantly reduce the computation time needed to find a solution.
- We would like to explore this empirically in the **8-puzzle** example.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

3. Heuristic Search – Empirical Comparison of the Search Algorithms

- For the 8-puzzle there are **two** simple admissible **heuristics**.
- The **heuristic h_1** simply counts the number of squares that are not in the right place. Clearly this heuristic is **admissible**.
- **Heuristic h_2** measures the *Manhattan distance*. For every square the **horizontal** and **vertical** distances to that's square's location in the goal state are added together.

2	5	
1	4	8
7	3	6

and

1	2	3
4	5	6
7	8	

- This value is then summed over all squares.
- For example, the Manhattan distance of the two states is calculated as
$$h_2(s) = 1 + 1 + 1 + 1 + 2 + 0 + 3 + 1 = 10$$
- The **admissibility** of the Manhattan distance is also obvious.

3. Heuristic Search

Empirical Comparison of the Search Algorithms

- The described algorithms were implemented in *Mathematica*.
- For a comparison with uninformed search, the **A* algorithm** with the two heuristics h_1 and h_2 and **iterative deepening** was applied to 132 randomly generated 8-puzzle problems.
- The average values for the number of steps and computation time are given in **Table 4.3**.
- **Table 4.3** illustrates the comparison of the computation cost of **uninformed search** and **heuristic search** for solvable 8-puzzle problems with various depths. Measurements are in steps and seconds. *All values are averages over multiple runs* (see last column).
- We see that the **heuristics** significantly **reduce** the **search cost** compared to **uninformed search**.

Depth	Iterative deepening		A★ algorithm				Num. runs
	Steps	Time [sec]	Heuristic h_1		Heuristic h_2		
			Steps	Time [sec]	Steps	Time [sec]	
2	20	0.003	3.0	0.0010	3.0	0.0010	10
4	81	0.013	5.2	0.0015	5.0	0.0022	24
6	806	0.13	10.2	0.0034	8.3	0.0039	19
8	6455	1.0	17.3	0.0060	12.2	0.0063	14
10	50512	7.9	48.1	0.018	22.1	0.011	15
12	486751	75.7	162.2	0.074	56.0	0.031	12
IDA★							
14	–	–	10079.2	2.6	855.6	0.25	16
16	–	–	69386.6	19.0	3806.5	1.3	13
18	–	–	708780.0	161.6	53941.5	14.1	4

Table 4.3: Comparison of the computation cost of uninformed search and heuristic search

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

3. Heuristic Search

Empirical Comparison of the Search Algorithms

- If we compare **iterative deepening** to A* with h_1 at depth **12**, for example, it becomes evident that h_1 reduces the number of steps by a factor of about **3.000** (486751/162.2), but the computation time by only a factor of **1.023** (75.7/0.074). *This is due to the higher cost per step for the computation of the heuristic.*
- Closer examination reveals a jump in the number of steps between depth 12 and 14 in the column for h_1 . This jump cannot be explained solely by the repeated work done by IDA*.
- It comes about because the implementation of the A* algorithm deletes duplicates of identical nodes and thereby shrinks the search space.*
- This is not possible with IDA* because it saves almost no nodes. Despite this, A* can no longer compete with IDA* beyond depth 14 because the cost of **sorting** in new nodes pushes up the time per step so much.

Depth	Iterative deepening		A★ algorithm				Num. runs
	Steps	Time [sec]	Heuristic h_1		Heuristic h_2		
			Steps	Time [sec]	Steps	Time [sec]	
2	20	0.003	3.0	0.0010	3.0	0.0010	10
4	81	0.013	5.2	0.0015	5.0	0.0022	24
6	806	0.13	10.2	0.0034	8.3	0.0039	19
8	6455	1.0	17.3	0.0060	12.2	0.0063	14
10	50512	7.9	48.1	0.018	22.1	0.011	15
12	486751	75.7	162.2	0.074	56.0	0.031	12
IDA★							
14	–	–	10079.2	2.6	855.6	0.25	16
16	–	–	69386.6	19.0	3806.5	1.3	13
18	–	–	708780.0	161.6	53941.5	14.1	4

Table 4.3: Comparison of the computation cost of uninformed search and heuristic search

3. Heuristic Search

Empirical Comparison of the Search Algorithms

- A computation of the effective branching factor, according to **Theorem 1**, yield values of about 2.8 for **uninformed search**.
- Heuristic h_1 reduces the branching factor to values of about **1.5** and h_2 to about **1.3**.
- We can see in the table that a small reduction of the branching factor from 1.5 to 1.3 gives us a big **advantage in computation time**.
- *Heuristic search thus has an important practical significance because it can solve problems which are far out of reach for uninformed search.*

Depth	Iterative deepening		A★ algorithm				Num. runs
	Steps	Time [sec]	Heuristic h_1		Heuristic h_2		
			Steps	Time [sec]	Steps	Time [sec]	
2	20	0.003	3.0	0.0010	3.0	0.0010	10
4	81	0.013	5.2	0.0015	5.0	0.0022	24
6	806	0.13	10.2	0.0034	8.3	0.0039	19
8	6455	1.0	17.3	0.0060	12.2	0.0063	14
10	50512	7.9	48.1	0.018	22.1	0.011	15
12	486751	75.7	162.2	0.074	56.0	0.031	12
			IDA★				
14	–	–	10079.2	2.6	855.6	0.25	16
16	–	–	69386.6	19.0	3806.5	1.3	13
18	–	–	708780.0	161.6	53941.5	14.1	4

Table 4.3: Comparison of the computation cost of uninformed search and heuristic search

3. Heuristic Search - Summary

- Of the various search algorithms for *uninformed* search, **iterative deepening** is the only practical one because it is **complete** and can get by with **very little memory**.
- *However, for difficult combinatorial search problems, even iterative deepening usually fails due to the size of the search space.*
- **Heuristic search** helps here through its **reduction** of the **effective branching factor**.
- The **IDA* algorithm**, like **iterative deepening**, is **complete** and requires **very little memory**.
- Heuristics naturally only give a significant advantage if the heuristic is “**good**”.
- When solving difficult search problems, the developer’s actual task consist of designing heuristics which greatly **reduce** the **effective branching factor**.
- In closing, it remains to note that **heuristics** have no performance advantage for **unsolvable problems** because the unsolvability of a problem can only be established when the **complete search tree** has been searched through.
- *In summary we can say the following: for **solvable problems**, heuristics often reduce computation time dramatically, but for **unsolvable problems** the cost can be higher with heuristics.*

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

Contents

1. Introduction
2. Uninformed Search
3. Heuristic Search
- 4. Games with opponents**
5. Heuristic Evaluation Functions
6. State of the Art

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

4. Games with opponents

- **Games for two players**, such as **chess**, **checkers**, **Othello** and **Go** are **deterministic** because every action (a move) results in the same child state given the same parent state.
- In contrast, **backgammon** is **non-deterministic** because its child state depends on the result of a **dice roll**.
- These games are all **observable** because every player always knows the complete game state.
- Many **card games**, such as poker, for example, are only **partially observable** because the player does not know the other player's cards, or only has partial knowledge about them.
- The problems discussed so far were **deterministic** and **observable**.
- *In the following we will look at games which, too, are **deterministic** and **observable**.*
- Furthermore, we will limit ourselves to **zero-sum games**. These are games in which every gain one player makes means a loss of the same value for the opponent.
- *The sum of the gain and loss is always equal to zero.*
- This is true of the games **chess**, **checkers**, **Othello** and **Go** mentioned above.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

4. Games with opponents – Minimax search

- The **goal** of each player is to make optimal moves that result in victory.
 - In principle it is possible to construct a **search tree** and completely search through it (like with the 8-puzzle) for a series of moves that will result in victory.
 - However, there are several peculiarities to watch out for:
1. The **effective branching factor** in chess is around **30** to **35**. In a typical game with 50 moves per player, the search tree has more than $30^{100} \approx 10^{148}$ leaf nodes. *Thus there is no chance to fully explore the search tree!*

Additionally, chess is often played with a **time limit**.

Because of the **real-time requirement**, the search must be **limited** to an appropriate **depth** in the tree, for example eight half-moves.

Since among the leaf nodes of this **depth-limited tree** there are normally **no solution nodes** (that is, nodes which terminate the game), a **heuristic evaluation function** B for board positions is used.

The level of play of the program strongly depends on the quality of this evaluation function.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

4. Games with opponents – Minimax search

- However, there are several peculiarities to watch out for:
2. In the following we will call the player whose game we wish to optimize **Max** and his opponent **Min**. The opponent's (Min's) moves are not known in advance, and thus neither is the actual search tree. This problem can be elegantly solved by assuming that the opponent always makes the best move he can.
The higher the evaluation $B(s)$ for position s , the better position s is for the player **Max** and the worse it is for the opponent **Min**.
Max tries to maximize the evaluation of his moves, whereas **Min** makes moves that result in as low an evaluation as possible.

4. Games with opponents – Minimax search

- A search tree with four half-moves and evaluations of all leaves is given in **Figure 4.20**.
- The evaluation of an inner node is derived **recursively** as the **maximum** of **minimum** of its child nodes, depending on the node's level.

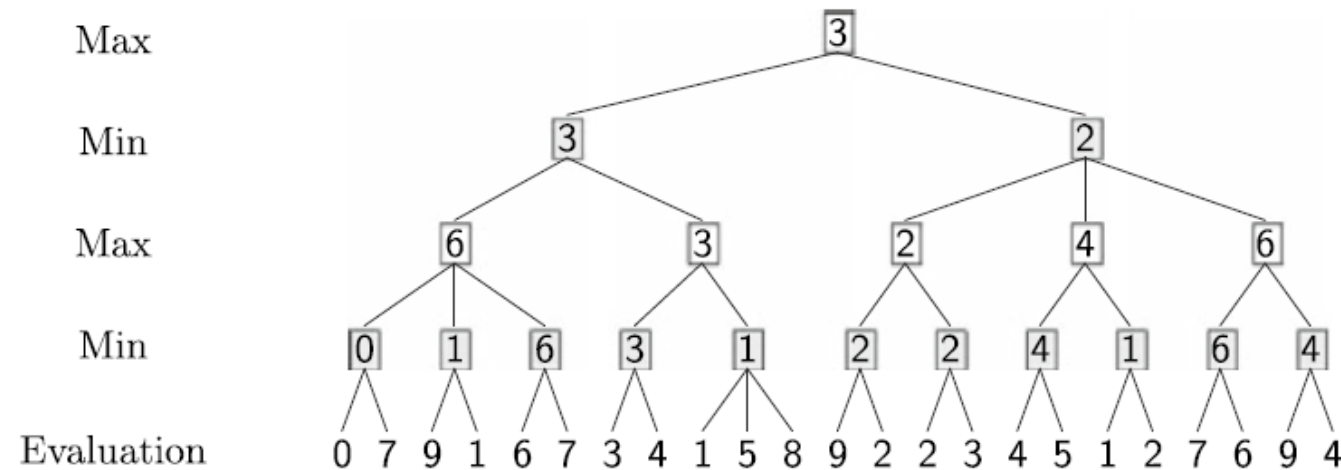


Figure 4.20: A minimax game tree with look-ahead of four halve-moves

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

4. Games with opponents – Alpha-Beta-Pruning

- By switching between maximization and minimization, we can save ourselves a lot of work in some circumstances.
- **Alpha-beta pruning** works with **depth-first search** up to a preset **depth limit**.
- In this way, the search tree is searched through **from left to right**.
- Like in minimax search, in the **minimum nodes** the **minimum** is generated from the **minimum value of the successor nodes** and in the **maximum nodes** likewise the **maximum**.

- In **Figure 4.21** this process is depicted for the tree from **Figure 4.20**. The dotted portions of the tree are not traversed because they have no effect on the end result.
- At the node marked ***a***, all other successors can be ignored after the first child is evaluated as the value 1, because the **minimum** is sure to be ≤ 1 .
- It could even become smaller still, but that is irrelevant since the maximum is already ≥ 3 one level above.

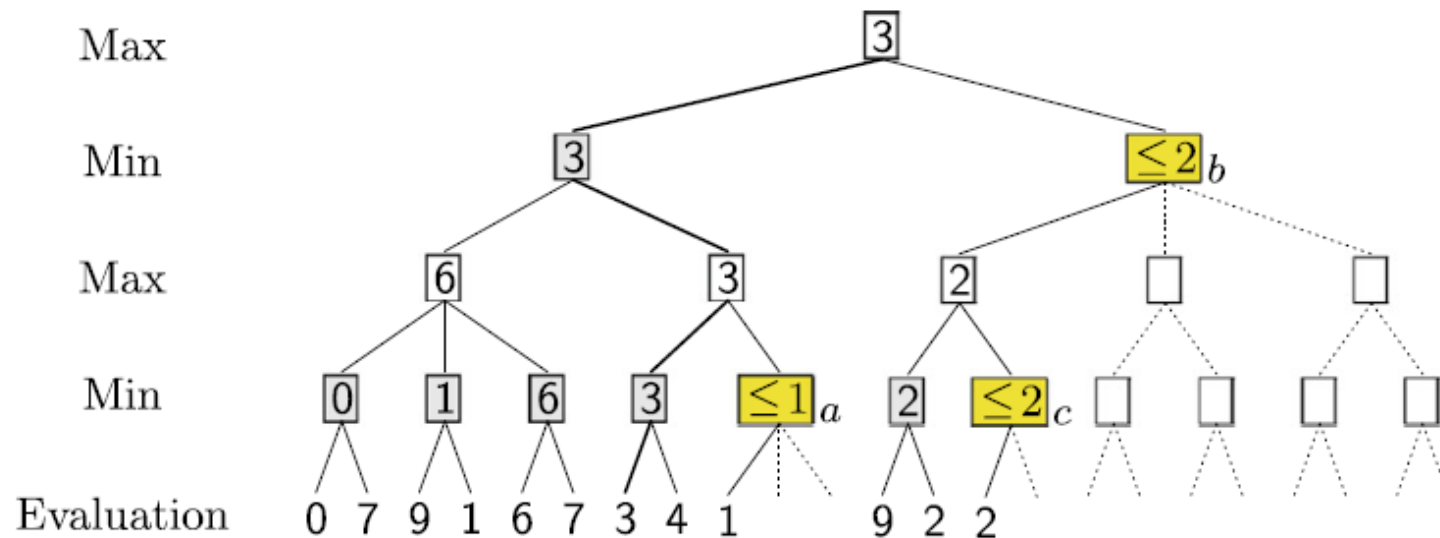


Figure 4.21: An alpha-beta game tree with look-ahead of four half moves

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

4. Games with opponents – Alpha-Beta-Pruning

- Regardless of how the evaluation of the remaining successors turns out, the **maximum** will keep the value 3.
- Analogously the tree will be trimmed at node **b**.
- Since the first child of **b** has the value 2, the minimum to be generated for **b** can only be less than or equal to 2. But the maximum at the root node is already sure to be ≥ 3 . This cannot be changed by values ≤ 2 . Thus the remaining subtrees of **b** can be pruned.

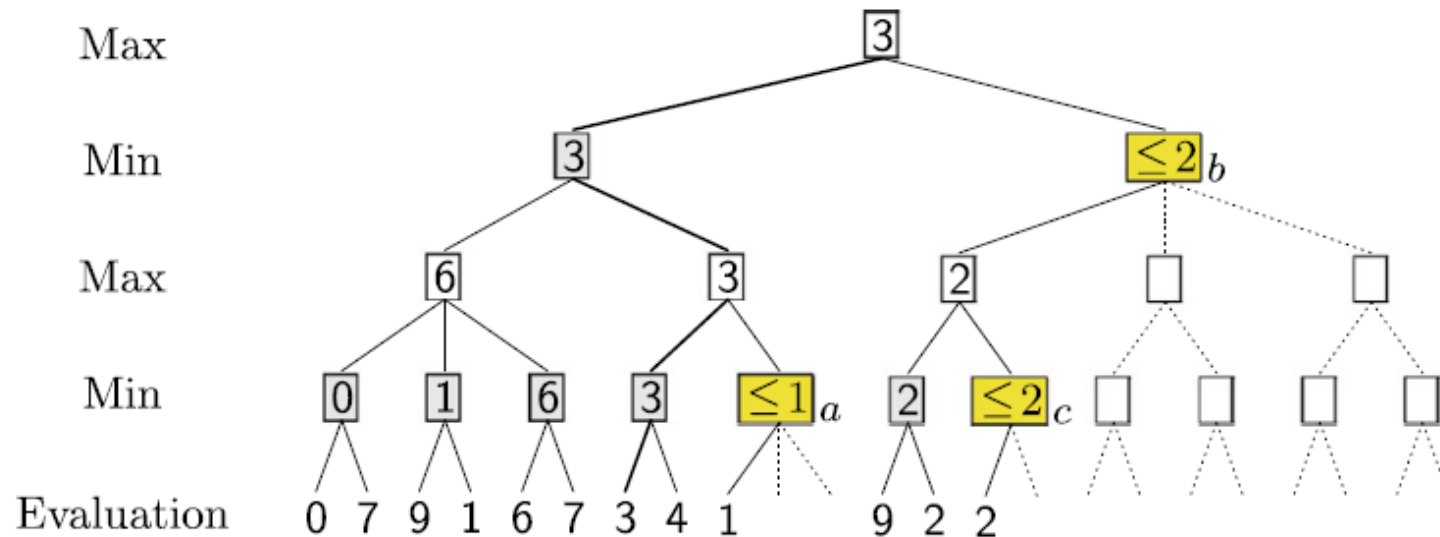


Figure 4.21: An alpha-beta game tree with look-ahead of four half moves

4. Games with opponents – Alpha-Beta-Pruning

- The same reasoning applies for the node **c**.
- However, the relevant **maximum** node is not the direct *parent*, but the **root node**. This can be generalized.
- At every leaf node the evaluation is calculated.
- For every **maximum** node the current *largest* child value is saved in α .
- For every **minimum** node the current *smallest* child value is saved in β .
- *If at a **minimum** node k the current value $\beta \leq \alpha$, then the search under k can **end**. Here α is the largest value of a maximum node in the path from the root to k .*
- *If at a **maximum** node l the current value $\alpha \geq \beta$, then the search under l can end. Here β is the smallest value of a minimum node in the path from the root to l .*

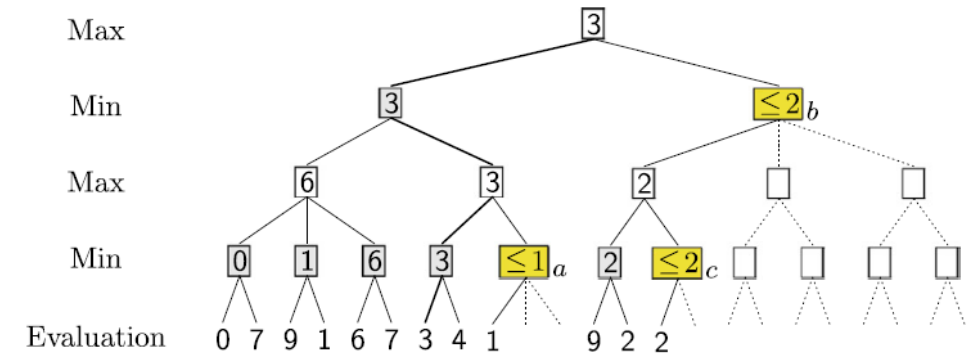


Figure 4.21: An alpha-beta game tree with look-ahead of four half moves

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

4. Games with opponents – Alpha-Beta-Pruning

- The algorithm given in **Figure 4.22** is an extension of **depth-first search** with **two functions** which are called in *alternation*. It uses the values defined for α and β .

```
ALPHABETAMAX(Node,  $\alpha$ ,  $\beta$ )  
If DepthLimitReached(Node) Return(Rating(Node))  
NewNodes = Successors(Node)  
While NewNodes  $\neq \emptyset$   
     $\alpha$  = Maximum( $\alpha$ , ALPHABETAMIN(First(NewNodes),  $\alpha$ ,  $\beta$ ))  
    If  $\alpha \geq \beta$  Return( $\beta$ )  
    NewNodes = Rest(NewNodes)  
Return( $\alpha$ )
```

```
ALPHABETAMIN(Node,  $\alpha$ ,  $\beta$ )  
If DepthLimitReached(Node) Return(Rating(Node))  
NewNodes = Successors(Node)  
While NewNodes  $\neq \emptyset$   
     $\beta$  = Minimum( $\beta$ , ALPHABETAMAX(First(NewNodes),  $\alpha$ ,  $\beta$ ))  
    If  $\beta \leq \alpha$  Return( $\alpha$ )  
    NewNodes = Rest(NewNodes)  
Return( $\beta$ )
```

Figure 4.22: The algorithm for alpha-beta search with the two functions AlphaBetaMin and AlphaBetaMax

- The initial alpha-beta pruning call is done with the command

$\text{AlphaBetaMax}(\text{RootNode}, -\infty, +\infty)$

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

4. Games with opponents – Alpha-Beta-Pruning

- Complexity
- The **computation time** saved by alpha-beta pruning heavily depends on the **order** in which child nodes are traversed.
- *In the worst case, alpha-beta pruning does not offer any advantage.*
- For a constant branching factor b the number n_d of leaf nodes to evaluate at depth d is equal to $n_d = b^d$
- **In the best case**, when the successors of **maximum** nodes are **decendingly** sorted and the successors of **minimum** nodes are **ascendingly** sorted, the effective branching factor is reduced to $\sqrt{b}$.
- In **chess** this means a substantial reduction of the effective branching factor from **35** to about **6**.
- Then only $n_d = \sqrt{b^d} = b^{\frac{d}{2}}$ leaf nodes would be created.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

4. Games with opponents – Alpha-Beta-Pruning

- Complexity
- This means that the **depth limit** and thus also the **search horizon** are **doubled** with **alpha-beta pruning**.
- However, this is only true in the case of **optimally sorted successors** because the child nodes' ratings are unknown at the time when they are created.
- If the child nodes are **randomly** sorted, then the branching factor is reduced to $b^{\frac{3}{4}d}$
- To double the search depth as mentioned above, we would need the child nodes to be optimally ordered, which is not the case in practice. Otherwise the search would be unnecessary.
- With a simple trick we can get a relatively good node ordering.
- *We connect alpha-beta pruning with iterative deepening over the depth limit.*
- Thus at every new depth limit we can access the ratings of all nodes of previous levels and order the successors at every branch.
- Thereby we reach an **effective branching factor** of roughly **7** to **8**, which is not far from the theoretical optimum of $\sqrt{35}$.

4. Games with opponents – Non-deterministic Games

- **Minimax search** can be generalized to all games with **non-deterministic** actions, such as backgammon.
- Each player rolls before his move, which is influenced by the result of the dice roll.
- In the game tree there are now therefore **three types of levels** in the sequence
*Max, **dice**, Min, **dice**, ...*
- Where each dice roll node branches six ways.
- Because we cannot predict the value of the die, we **average** the values of all rolls.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

Contents

1. Introduction
2. Uninformed Search
3. Heuristic Search
4. Games with opponents
- 5. Heuristic Evaluation Functions**
6. State of the Art

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

5. Heuristic Evaluation Functions

- *How do we find a good heuristic evaluation function for the task of searching?*
- Here there are fundamentally **two** approaches.
- The classical way uses the **knowledge** of **human experts**.
- The **knowledge engineer** is given the usually difficult task of **formalizing** the expert's implicit knowledge in the form of a **computer program**.
- We now want to show how this process can be simplified in the **chess** program example.

5. Heuristic Evaluation Functions

- In the **first step**, experts are questioned about the most important factors in the selection of a move.
- Then it is attempted to quantify these factors.
- We obtain a list of *relevant features* or **attributes**.
- These are then (in the simplest case) combined into a **linear evaluation function** $B(s)$ for **positions**, which could look like:

$$\begin{aligned} B(s) &= a_1.\text{material} + a_2.\text{pawn_structure} + a_3.\text{king_safety} + a_4.\text{knight_in_center} \\ &+ a_5.\text{bishop_diagonal_coverage} + \dots [4.3] \end{aligned}$$

- where **material** is by far the most important feature and is calculated by
$$\text{material} = \text{material}(\text{own}_{\text{team}}) - \text{material}(\text{opponent})$$

- with

$$\begin{aligned} \text{material}(\text{team}) &= \text{num_pawns}(\text{team}).\mathbf{100} + \text{num_knights}(\text{team}).\mathbf{300} + \text{num_bishops}(\text{team}).\mathbf{300} \\ &+ \text{num_rooks}(\text{team}).\mathbf{500} + \text{num_queens}(\text{team}).\mathbf{900} \end{aligned}$$

- Nearly all **chess** programs make a similar evaluation for material.

5. Heuristic Evaluation Functions

- However, there are big differences of all other features, we will not go into here.
- In the **next step** the weight a_i of alle features must be determined.
- These are set *intuitively* after discussion with **experts**, then *changed* after each game based of **positive** and **negative** experience.
- The fact that this optimization process is very expensive and furthermore that the linear combination of features is very limited suggest the use of **machine learning**.

5. Heuristic Evaluation Functions – Learning of Heuristics

- We now want to automatically optimize the **weights** a_i of the **evaluation function** $B(s)$ in [4.3].
- In this approach the **expert** is only asked about the relevant features $f_1(s), \dots, f_n(s)$ for game state s .
- Then a **machine learning process** is used with the **goal** of finding an evaluation function that is as close to optimal as possible.
- We start with an **initial pre-set evaluation function** (determined by the learning process) and then let the chess program play.
- At the end of the game a **rating** is derived from the result (*victory, defeat, or draw*).
- Based on this rating, the evaluation function is *changed* with the goal of making fewer mistakes next time.
- In principle, the same thing that is done by the developer is now being taken care of automatically by the **learning process**.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

5. Heuristic Evaluation Functions – Learning of Heuristics

- As easy as this sounds, *it is very difficult in practice*.
- A central problem with improving the position rating based on *won* or *lost* matches is known today as the **credit assignment problem**.
- URL: [Assignment Problem](#)
- We do in fact have a rating at the end of the game, but no ratings for the **individual** moves.
- Thus the agent carries out many actions but does not receive any positive or negative feedback until the very end.
- *How should it then assign this feedback to the many actions taken in the past?*
- *And how should it improve its actions in that case?*
- The exciting field of **reinforcement learning** deals with these questions.
- URL: [Reinforcement Learning](#)

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

5. Heuristic Evaluation Functions – Learning of Heuristics

- **Monte Carlo tree search** (MCTS) works quite similarly (URL: [Monte Carlo tree search](#))
- To improve the heuristic rating of a game state s , a **random number of search tree branches** starting from this state are either explored to the end and evaluated, or stopped at a certain depth and then the leaf nodes are evaluated heuristically.
- The evaluation $B(s)$ of state s is given as the **mean** of all leaf node scores.
- The use of MCTS paths requires only a small part of the entire exponentially exploding tree to be searched. This is illustrated in **Figure 4.23**. In this sketch of a search tree, several MCTS paths to leaf nodes are shown in **red**. *Notice that only a small part of the tree is searched.*

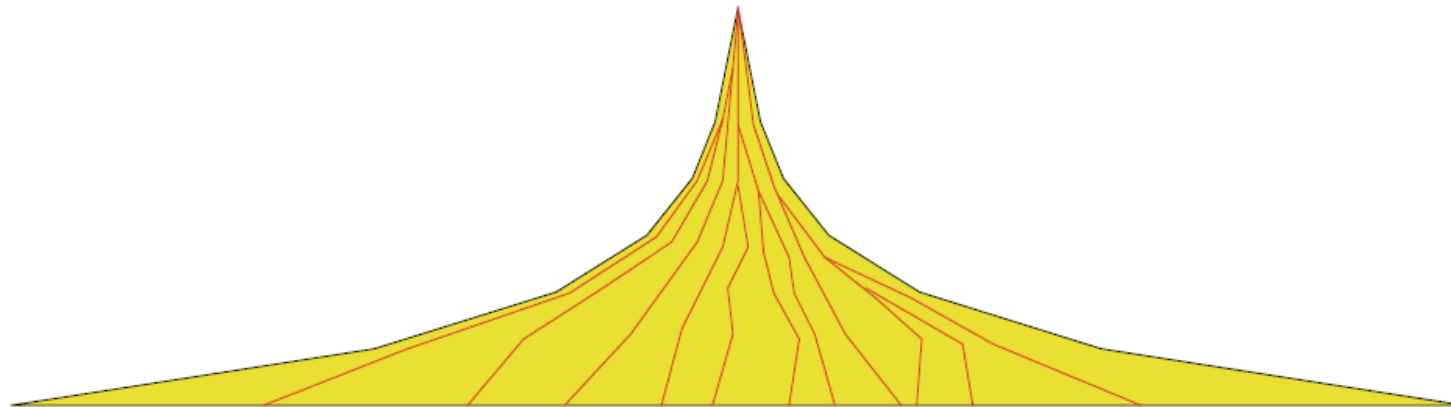


Figure 4.23: Monte Carlo tree search

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

5. Heuristic Evaluation Functions – Learning of Heuristics

- For many computer-simulated games, such as **chess**, this algorithm can be used to achieve better play for **the same computational effort** or **to reduce computational effort for the same difficulty level**.
- This method was used together with machine learning algorithms in 2016 by the program **AlphaGo**, which was the first **Go** program to defeat world-class human players.

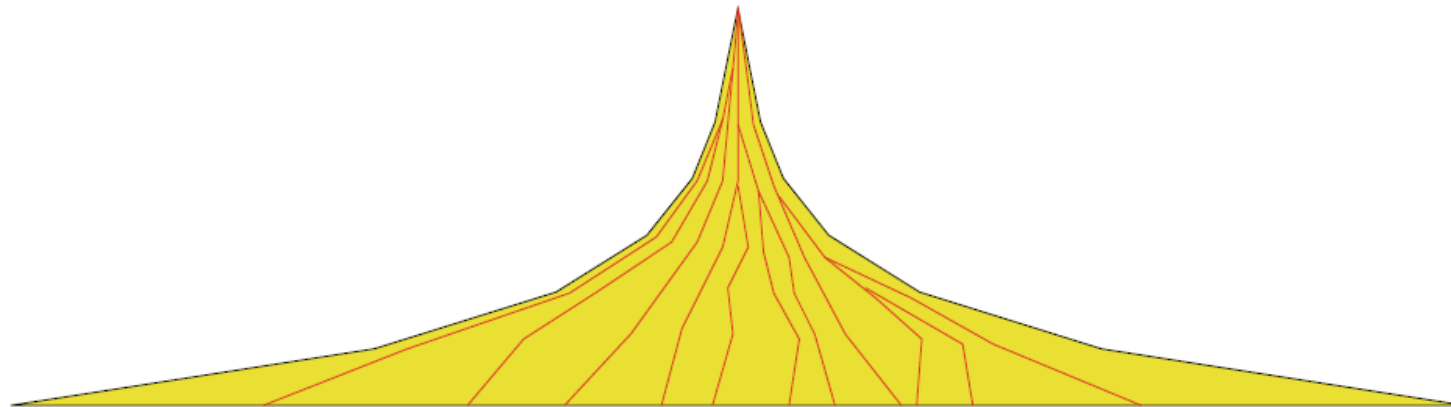


Figure 4.23: Monte Carlo tree search

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

Contents

1. Introduction
2. Uninformed Search
3. Heuristic Search
4. Games with opponents
5. Heuristic Evaluation Functions
- 6. State of the Art**

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

6. State of the Art

- For evaluation of the **quality** of the **heuristic search** processes, I would like to repeat **Elaine Rich's** definition [E. Rich. *Artificial Intelligence*. McGraw-Hill, 1983]:

Artificial Intelligence is the study of how to make computers do things at which, at the moment, people are better.

- There is hardly a better suited test for deciding whether a computer program is intelligent as the direct comparison of **computer** and **human** in a **game** like chess, checkers, backgammon or Go.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

6. State of the Art

- In **1950**, **Claude Shannon**, **Konrad Zuse** and **John von Neumann** introduced the first **chess** programs, which, however, could either not be implemented or would take a great deal of time to implement.
- Just a few years later, in **1955**, **Arthur Samuel** wrote a program that played **checkers** and could improve its own parameters through a **simple learning process**.
- To do this he used the first programmable logic computer, the **IBM 701**.
- *Compared to the chess computers of today, however, it had access to a large number of achieved games, for which every individual move had been rated by experts.*
- He solved the **credit assignment problem** in a simple manner.
- For each individual position during a game it compares the evaluation by the function $B(s)$ with the one calculated by **alpha-beta pruning** and changes $B(s)$ accordingly.
- In **1961** his **checkers** program beat the fourth-best checkers player in the USA.
- With this ground-breaking work, Samuel was surely nearly 30 years ahead of this time.
- Only at the beginning of the nineties, as **reinforcement learning** emerged, did **Gerald Tesauro** build a learning **backgammon** program named **TD-Gammon**, which played at the world champion level.

6. State of the Art – chess

- Today many **chess** programs exist that play above grandmaster level.
- The breakthrough came in **1997**, as IBM's **Deep Blue** defeated the chess world champion **Gary Kasparov** with a score of 3.5 games to 2.5.
- **Deep Blue** could on average compute **12 half-moves ahead** with **alpha-beta pruning** and **heuristic position evaluation**.
- Around the year **2005** one of the most powerful **chess** computers was **Hydra**, a parallel computer owned by a company in the United Arab Emirates.
- The software was developed by the scientists **Christian Donninger** (Austria) and **Ulf Lorenz** (Germany), as well as the German chess grand champion **Christopher Lutz**.
- **Hydra** uses 64 parallel Xeon processors with about 3 GHz computing power and 1 GByte memory each.
- For the position evaluation function each processor has an **FPGA** (Field Programmable Gate Array) co-processor. Thereby it becomes possible to evaluate **200 million positions per second** even with an expensive evaluation function.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

6. State of the Art – chess

- With this technology **Hydra** can on average compute about **18 moves ahead**.
- In special, critical situations, the search horizon can even be stretches out to *40 half-moves*.
- Clearly this kind of horizon is beyond what even grand champions can do, for Hydra often makes moves grand champions cannot comprehend, but which in the end lead to victory.
- In **2005 Hydra** defeated seventh ranked **grandmaster Michael Adams** with 5.5 – 0.5 games.
- Hydra uses little special textbook knowledge about chess, rather **alpha-beta search** with relatively general, well-known **heuristics** and a **good hand-coded position evaluation**.
- *In particular, **Hydra** is not capable of learning!*
- Improvements are carried out between games by the developers.
- As a consequence, **Hydra** was soon outperformed by machines that used **smart learning algorithms** rather than expensive hardware.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

6. State of the Art – chess

- In **2009** the system **Pocket Fritz 4**, running on a PDA, won the **Copa Mercusor chess** tournament in Buenos Aires with nine wins and one draw against 10 excellent human chess players, three of them grandmasters.
- Even though not much information about the internal structure of the software is available, this chess machine represents a trend away from raw computing power toward more intelligence.
- This machine plays at grandmaster level, and is comparable to, if not better than **Hydra**.
- According to Pocket Fritz developer **Stanislav Tsukrov**, Pocket Fritz with its chess search engine **HIARCS 13** searches less than *20.000 positions per second*, which is slower than Hydra by a factor of about 10.000.
- This leads to the conclusion that **HIARCS 13** definitely uses **better heuristics** to *decrease the effective branching factor* than Hydra and can thus well be called *more intelligent* than Hydra.
- By the way, **HIARCS** is a short for *Higher Intelligence Auto Response Chess System*.

6. State of the Art – Go

- Even though today no human stands a chance against the best **chess** computers, there are still many challenges for AI, for example **Go**.
- In this ancient Japanese game, played on a square board of 361 spaces with 181 white and 180 black stones, the **effective branching factor** is about **250**.
- After 8 half-moves there are already $1.5 \cdot 10^{19}$ possible positions.
- Given this complexity, none of the classic, well-known game tree search algorithms have a chance against a good human Go player.

AI Fundamentals

Chapter 4: Search, Games and Problem Solving

6. State of the Art – Go

- Yet in the most recent previous edition of this book, it was stated that:

*The experts agree that “truly intelligent” algorithms are needed here. Combinatoric enumeration is the **wrong approach**. Rather, procedures are needed that **recognize patterns** on the board, **track gradual developments**, and make **rapid “intuitive” decisions**.*

Similar to object recognition in complex images, we humans are still far superior to today’s computer programs. We process the image as a whole in a highly parallel manner, whereas the computer processess the millions of pixels successively and had great difficulty recognizing the essentials in the abundance of pixels.

*The program “The Many Faces on Go” recognizes **1100 different patterns** and knows **200 different playing strategies**. All Go programs, however, still have great difficulty recognizing a group of stones is dead or alive, or where in between to classify them.*

AI Fundamentals

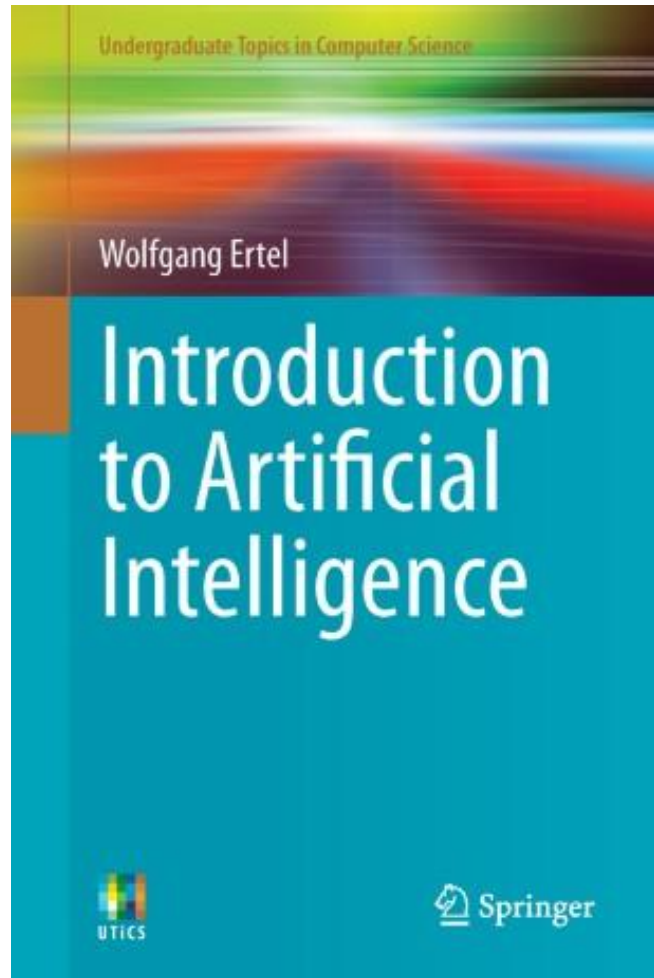
Chapter 4: Search, Games and Problem Solving

6. State of the Art – Go

- *This statement is now obsolete.*
- In January **2016**, **Google** and **Facebook** published the breakthrough concurrently.
- The same month, the program **AlphaGo**, developed by **Google DeepMind**, defeated European Go champion **Fan Hui** 5.0.
- Two months later, Korean player **Lee Sedol**, one of the best in the world, was defeated 4.1.
- **Deep Learning** for pattern recognition, **reinforcement learning** and **Monte Carlo tree search** lead to this successful result.
- The program plays hundreds of thousands of games against itself and uses the result (win, loss, draw) to learn the best possible heuristic score for a given position.
- **Monte Carlo tree search** (MCTS) is used as a replacement for **Minimax search**, which is not suitable for **Go**.

AI Fundamentals

End of Chapter 4 – Search, Games and Problem Solving



VIVES University of Applied Sciences
Bachelor in electronics-ICT

